Generative AI and Applications

Course Code: 25CS60TP1378-3

L:3 Hrs, P: 2 Hr, Per Week Total Credits:4

Department of Computer Science and Applications

By Dr. Suhashini Chawrasia

Unit 4

Self-Attention, The Problem of Average Meaning, Query, Key and Value Vectors in self-attention, Scaled Dot Product Attention, Multi-Head Attention

Self-Attention

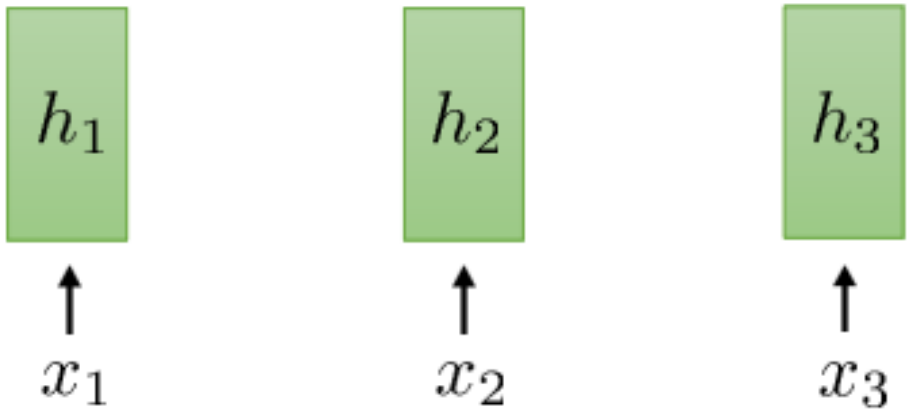
- Remove the recurrent dependency (no RNN) on the output of the previous timestep.
- The Sequential dependency is removed during training. Tokens are processes in parallel.

- Ashish Vaswani et al. (2017)
  - Don’t rely on the previous Decoder hidden state
  - Use hidden states from the same sequence to compute the attention score
  - Thus, each token attends to all other tokens in the same sequence
  - No need to wait for the generation of the previous token during training

Self-Attention

- Self-attention processes all the elements of a sequence in parallel without any regard for their order.
- Allows model to compute a weighted sum of the values in the sequence, where the weights are determined by the similarity between the query and key vectors.
- This allows the model to focus on different parts of the sequence when processing each token, enabling it to capture dependencies between token regardless of their distance in the sequence.

Self-Attention



hidden layer

$h_t = \sigma(Wx_t + b)$

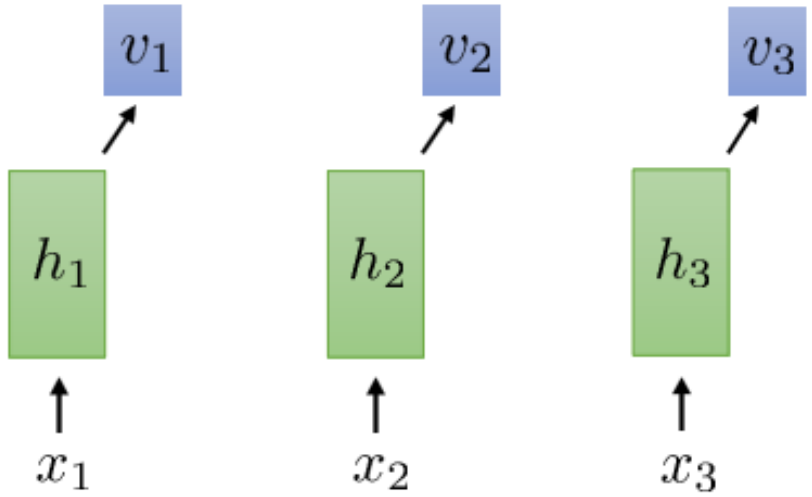
Query and Key Vectors

- To search a video on YouTube, user give certain description of video.
- It is an input vector that tells about what user wants.
- It serves as a Query.
- YouTube has vector representation of every video based on its properties.
- It serves as Key.
- The input query is matched against the keys and most relevant videos are fetched.

Query, Key, Value

- Query (q): Represents the current token's search intent or vector asking for context. It is matched against keys to determine how much attention to pay to other tokens in the sequence.
- Key (k): Represents the keyword or metadata vector of each token in the sequence. It acts as a keyword that gets compared against queries to compute relevance or similarity scores.
- Value (v): Contains the actual contextual information or feature representation of each token. It is weighted by the computed attention scores to produce the final aggregated output vector.

Self-Attention: compute value v_t



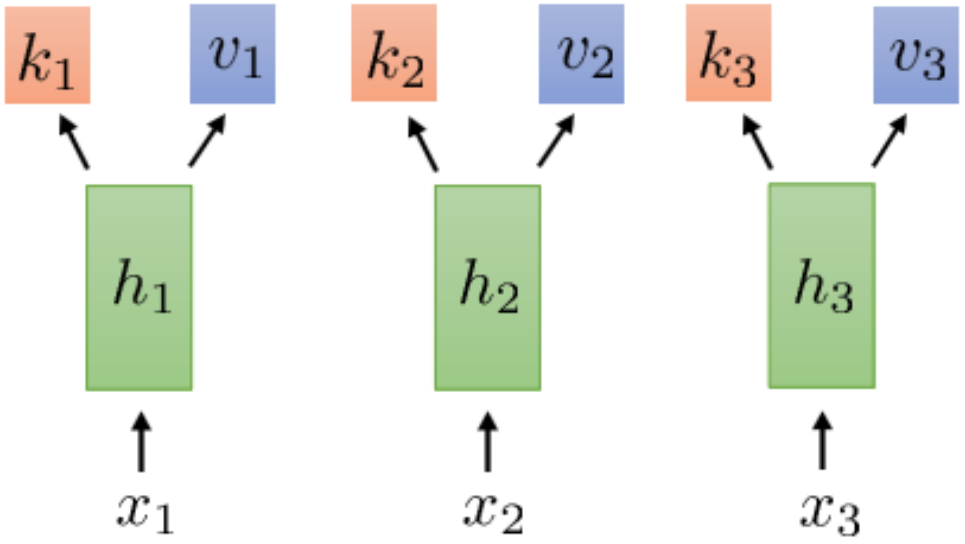
value

$v_t = W_v h_t$

hidden layer

$h_t = \sigma(W x_t + b)$

Self-Attention: compute key k_t



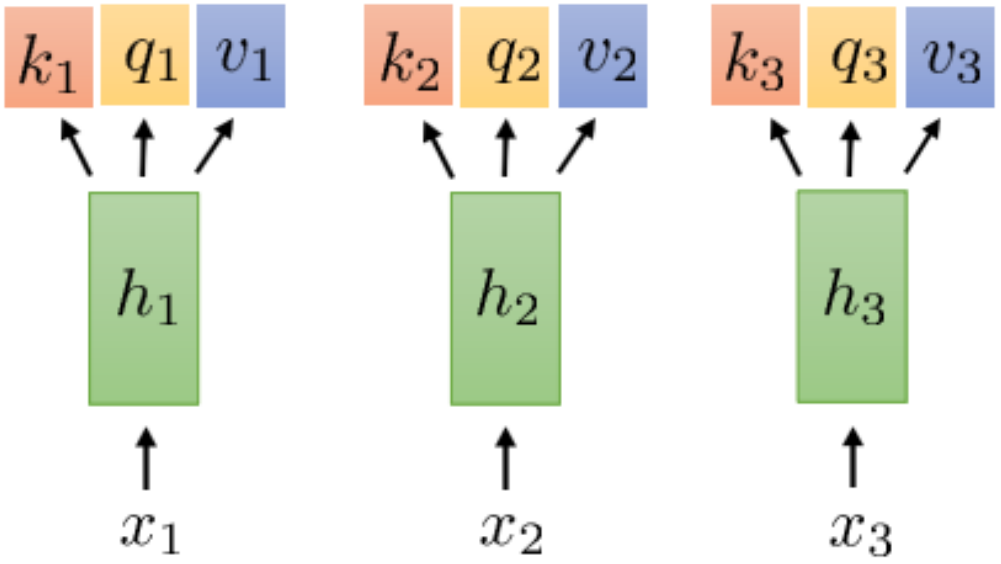
key, value

$$k_t = W_k h_t$$
$$v_t = W_v h_t$$

hidden layer

$$h_t = \sigma(W x_t + b)$$

Self-Attention: compute query q_t



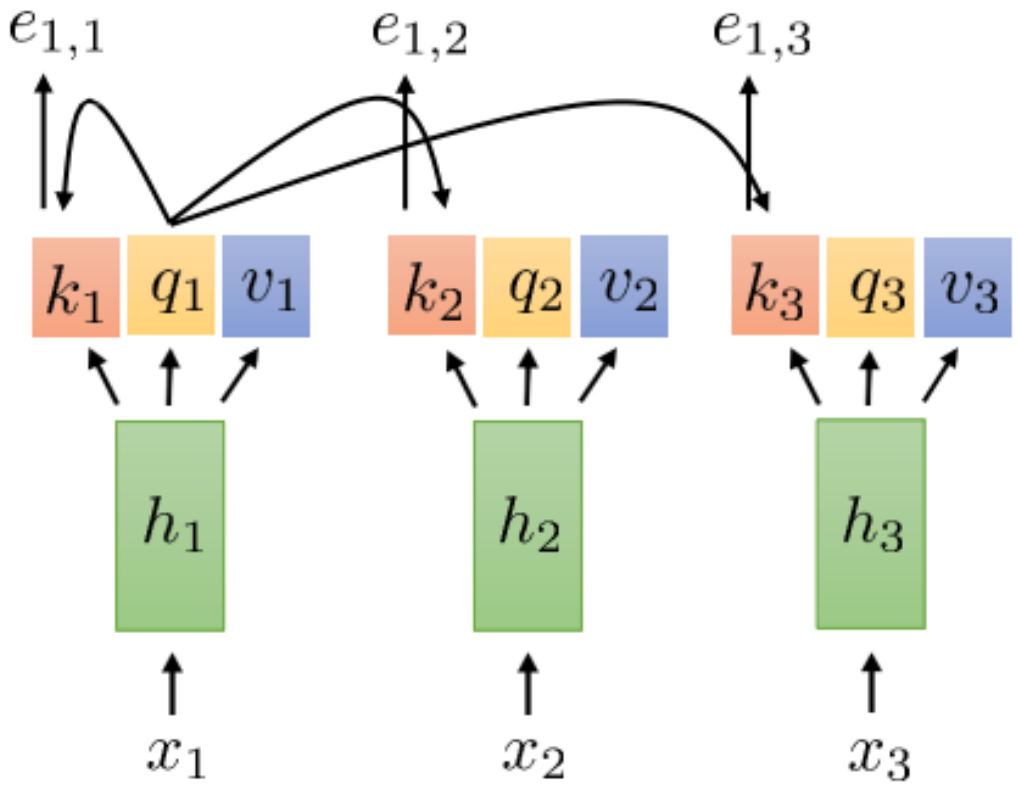
query, key, value

$$q_t = W_q h_t$$
$$k_t = W_k h_t$$
$$v_t = W_v h_t$$

hidden layer

$$h_t = \sigma(W x_t + b)$$

Self-Attention: Compute attention score $e_{l,t}$



attention score

$$e_{l,t} = q_l k_t$$

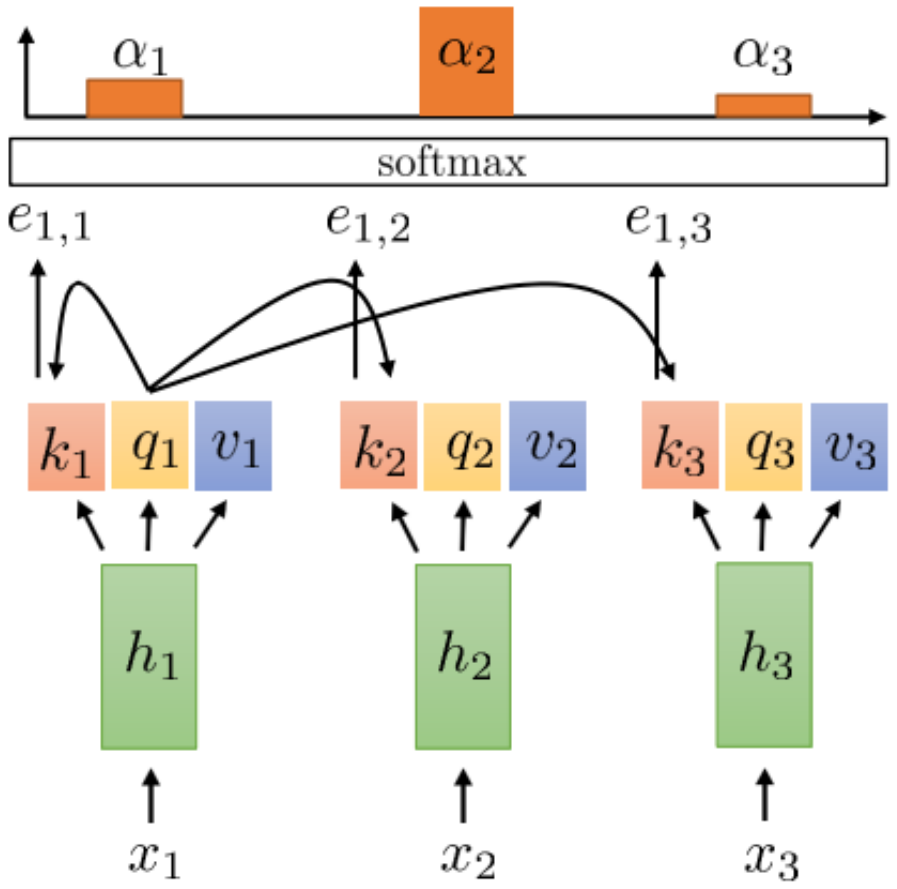
query, key, value

$$q_t = W_q h_t$$
$$k_t = W_k h_t$$
$$v_t = W_v h_t$$

hidden layer

$$h_t = \sigma(W x_t + b)$$

Self-Attention: Apply softmax to compute attention distribution $\alpha_{l,t}$



attention distribution

$$\alpha_{l,t} = softmax(e_{l,t}) = \frac{exp(e_{l,t})}{\sum_j \exp(e_{l,t})}$$

attention Score

$$e_{l,t} = q_l k_t$$

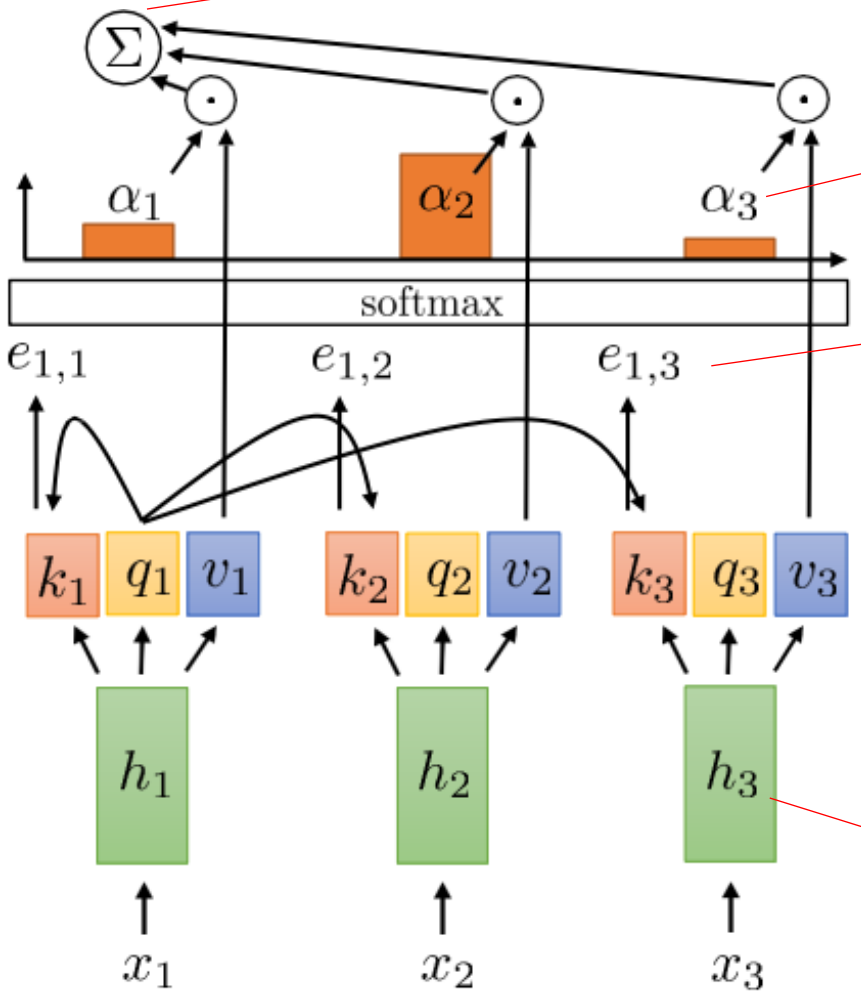
query, key, value

$$q_t = W_q h_t$$
$$k_t = W_k h_t$$
$$v_t = W_v h_t$$

hidden layer

$$h_t = \sigma(W x_t + b)$$

Self-Attention: compute attention output



attention output

$a_l = \sum_t \alpha_{l.t} v_t$

attention distribution

$\alpha_t = softmax(e_{l,t}) = \frac{\exp(e_{l,t})}{\sum_j \exp(e_{l,t})}$

attention Score

$e_{l,t} = q_l k_t$

query, key, value

$q_t = W_q h_t$

$k_t = W_k h_t$

$v_t = W_v h_t$

hidden layer

$h_t = \sigma(Wx_t + b)$

Assignment

Draw well-labelled diagram for the computation of Self-Attention scores for the second token t2

Scaled dot product Attention

- Attention score proposed by Vaswani et al. (2017) with extra scaling factor
- Scaled dot product attention refines the dot product attention by dividing the dot product by square root of the dimension of the key vector

$$\text{Attention}(q, k, v) = \text{softmax}\left(\frac{qk^T}{\sqrt{d^k}}\right)$$

$\sqrt{d^k}$ serves as a scaling factor

- Scaling helps in establishing the gradients across different dimensions of the key vector, which is useful when dealing with large dimensions and prevents the softmax from having extremely small gradients

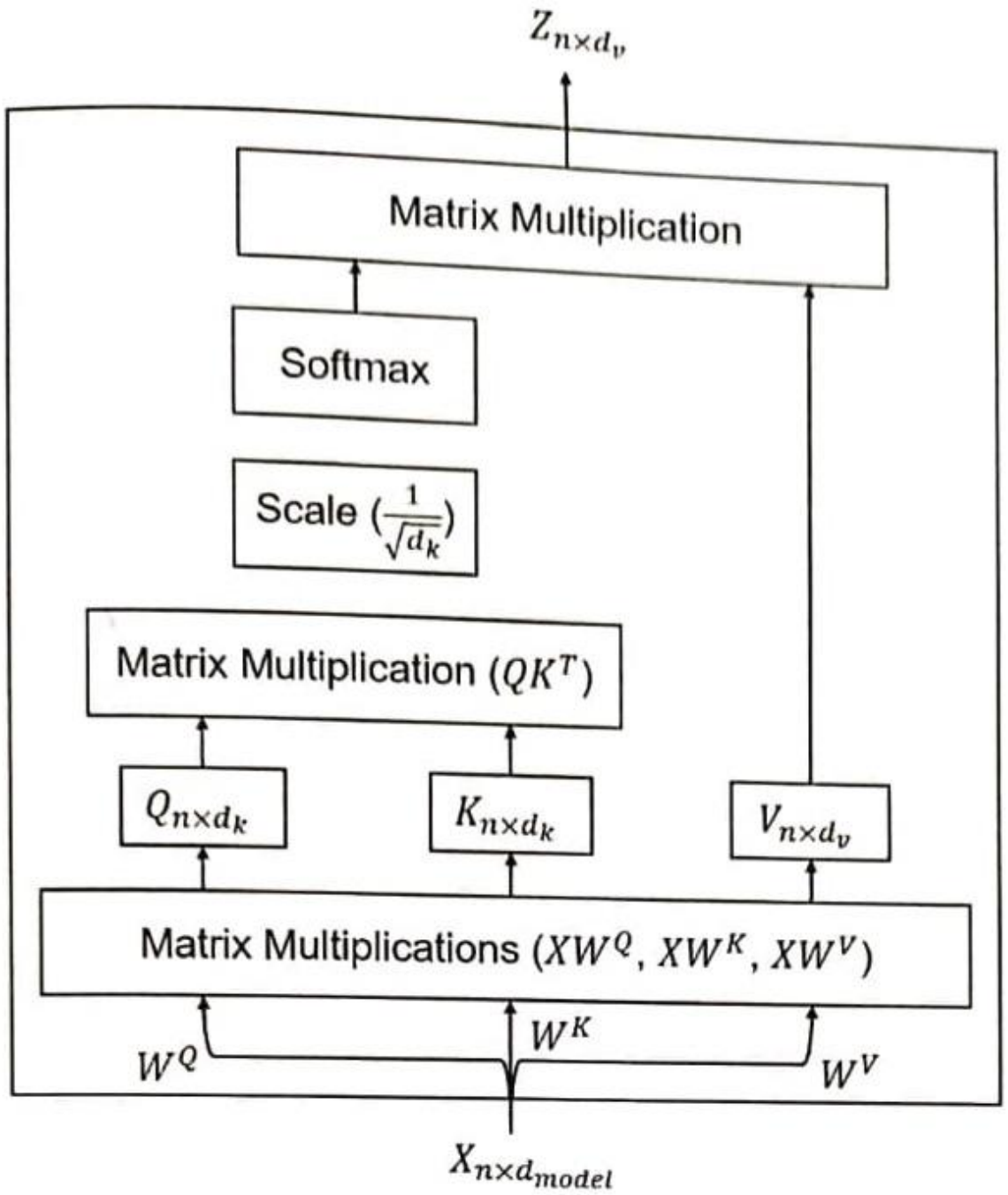
Final attention output

Self attention mechanism is a weighted sum for value vectors

$$\text{Attention}(q, k, v) = \text{softmax}\left(\frac{qk^T}{\sqrt{d^k}}\right)v$$

Complete computational flow of Scaled Dot-Product Attention, starting from the input embeddings and show how Query (Q), Key (K), Value (V), attention scores, attention weights, and the final output are generated.

Scaled dot product Attention



Step 1: Compute hidden layer

Let $x_1, x_2, ..x_N$ be the input sequence matrix
where
N is the number of tokens
$h_t = Wx_t + b$

Step 2: Compute query, key, value

q= query: current token

k =key: Relevant information the token contains to match queries

v= value: Contains the actual content to be aggregated

$q_t = W_q h_t$

$k_t = W_k h_t$

$v_t = W_v h_t$

Step 3: Compute attention scores

$$e_{l,t} = q_l k_t$$

Step 4: Scale dot product Attention

Attention score proposed by Vaswani et al. (2017) with extra scaling factor

Scaled dot product attention refines the dot product attention by dividing the dot product by square root of the dimension of the key vector

$$\left(\frac{1}{\sqrt{d^k}}\right)$$

$\sqrt{d^k}$ serves as a scaling factor

$$e_{l,t} = \left(\frac{q_l k_t}{\sqrt{d^k}}\right)$$

Scaling helps in establishing the gradients across different dimensions of the key vector, which is useful when dealing with large dimensions and prevents the softmax from having extremely small gradients

Step 5: Apply softmax to compute attention distribution

Apply the row-wise softmax function to turn real-valued scores into a valid probability distribution

$\alpha_{l,t}$ = softmax($e_{l,t}$) here, $e_{l,t}$ is attention score a variable

Formula for softmax(x)=$\frac{e^x}{\sum_{t'} e^x}$ here e^x is exponential(x)

Probability varies between 0 and 1

Step 6: Compute attention output

Multiply the attention distribution A by the Value V to compute the weighted sum of representation vectors:

$a_l = \alpha_{l,t} v_t$

Step 7: Concatenation

Finally, concatenate the attention output to the decoder hidden state

$[a_l, s_t]$

Numerical based on self-attention

Given q=[2,4], k=[1,0], v=[2,1], d_k=4 compute Attention(q, k, v)

Attention (q, k, v) =softmax $\left(\frac{qk}{\sqrt{d^k}}\right) v$

q.k = (2x1)+(4x0)=2

$\sqrt{d^k} = \sqrt{4} = 2$

Scale score = 2/2=1

Apply softmax (1)=1

Multiply by v

1x[2,1]=[2,1]

Attention(q, k, v)=[2,1]

Assignment

Given

Query vector (Q): [4, 2]

Key vector (K): [1, 3]

Value vector (V): [5, 10]

a) Apply the dot product operation to calculate the raw alignment score between the Query and the Key.

b) Final attention weight is 0.2. Apply weight to the Value vector to compute the model's final context representation

Solution

a) Raw Alignment Score (Dot Product)

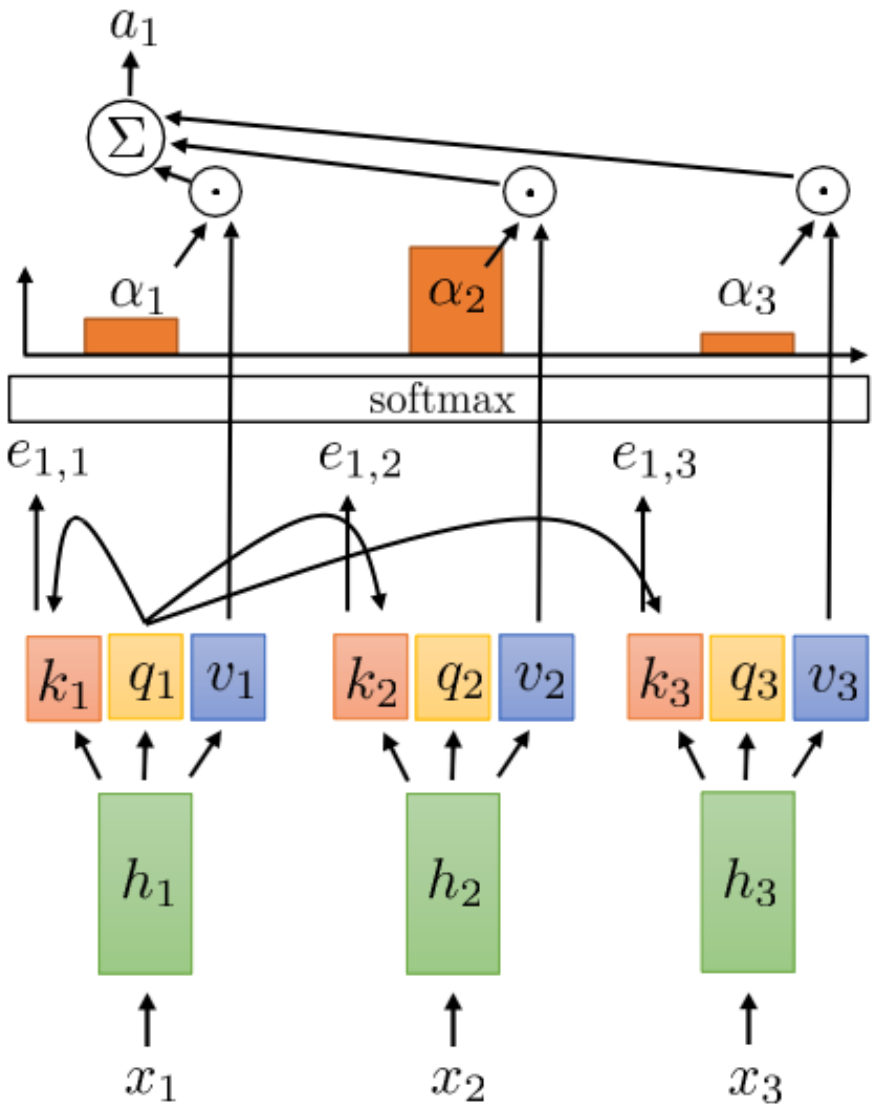
$qk = [4, 2]\ [1, 3] = 4 \times 1 + 2 \times 3 = 4 + 6 = 10$

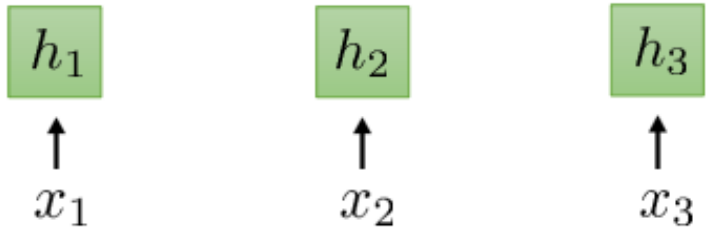
b) Final Context Representation

Applying the given attention weight of 0.2 to the value vector v:

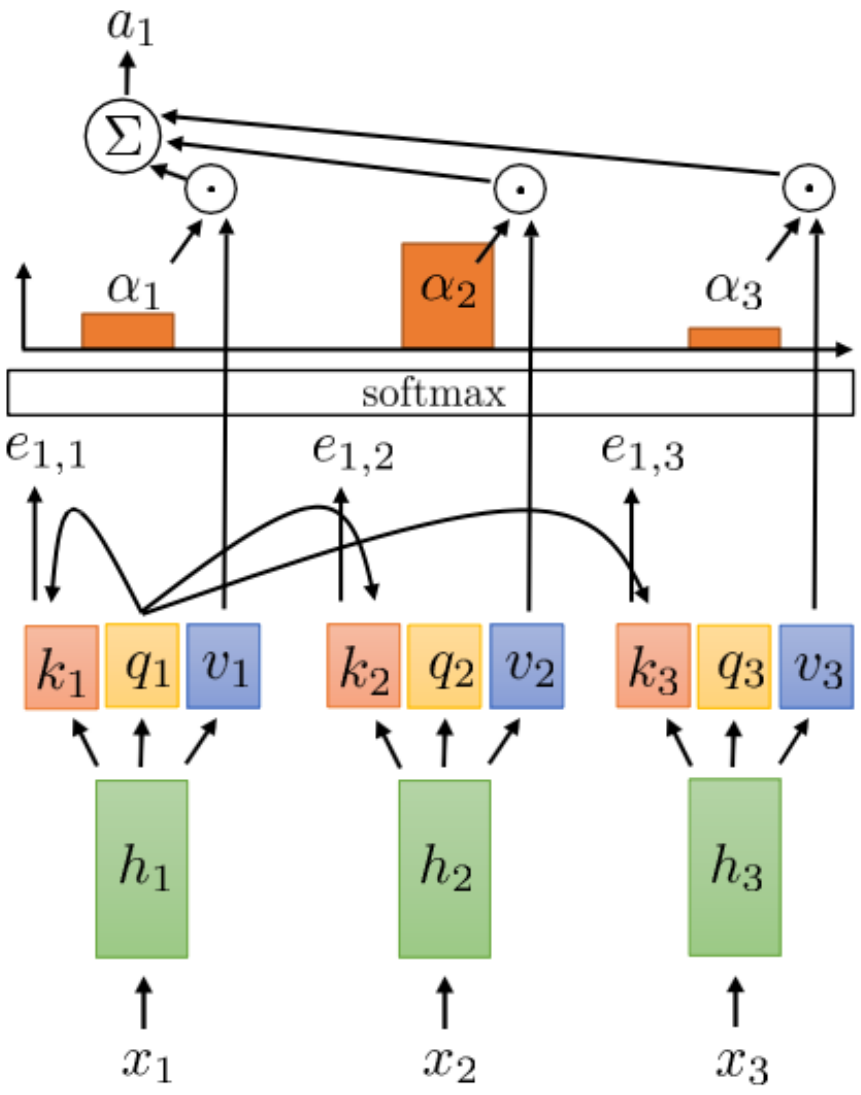
Context Representation $= 0.2 \times [5, 10] = [0.2 \times 5, 0.2 \times 10] = [1, 2]$

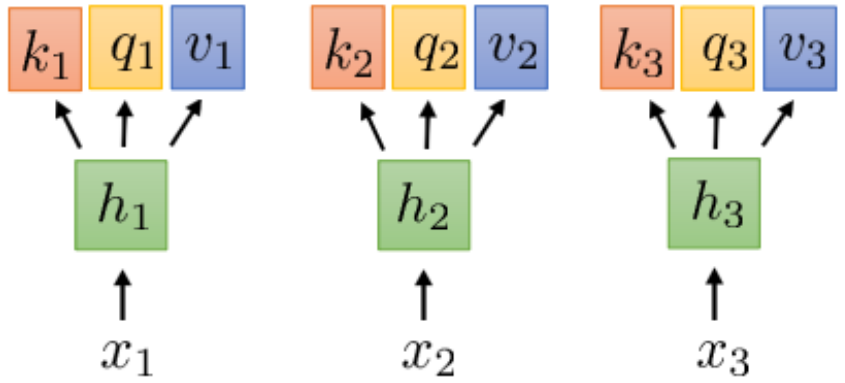
Self-Attention



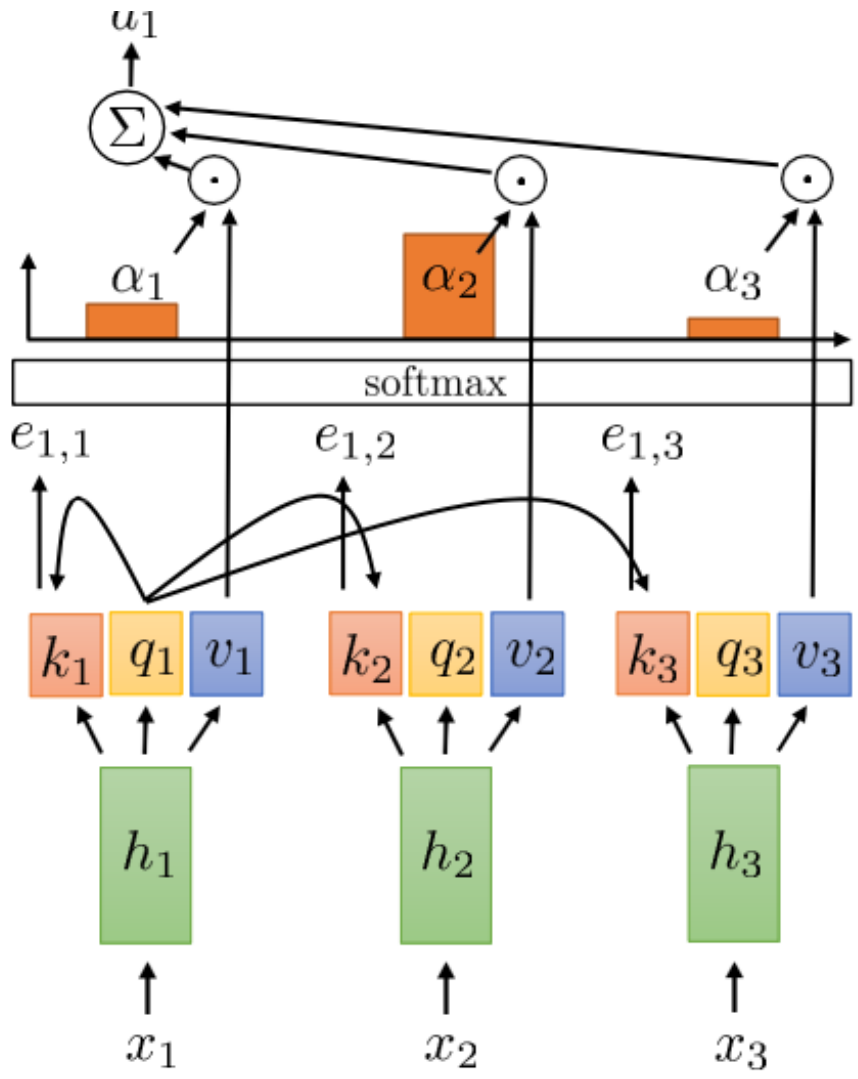


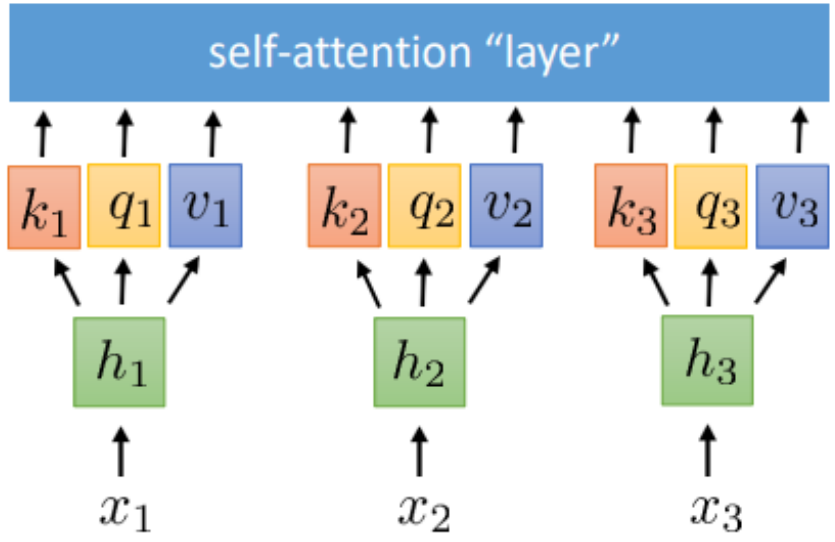
Self-Attention



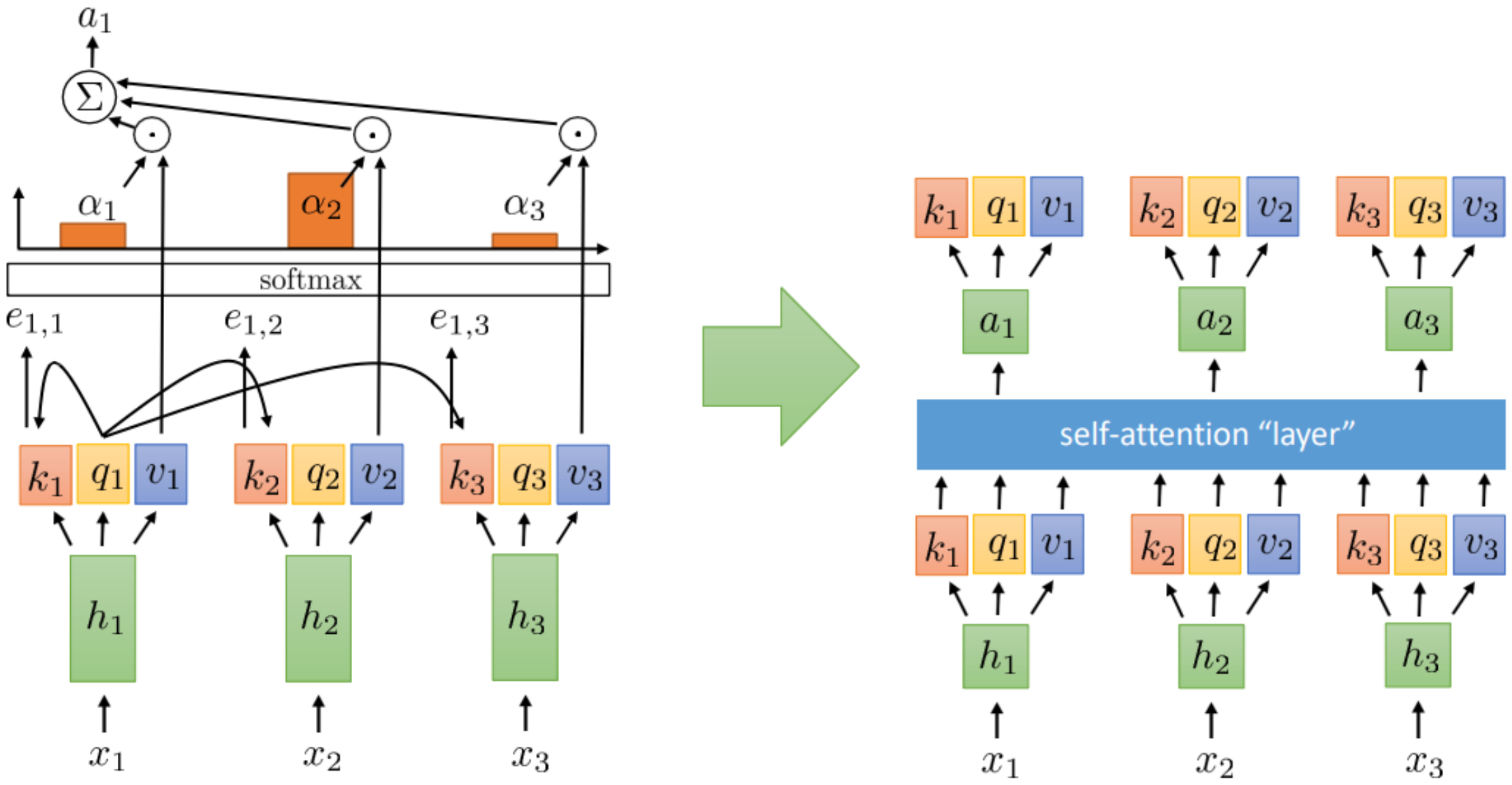


Self-Attention

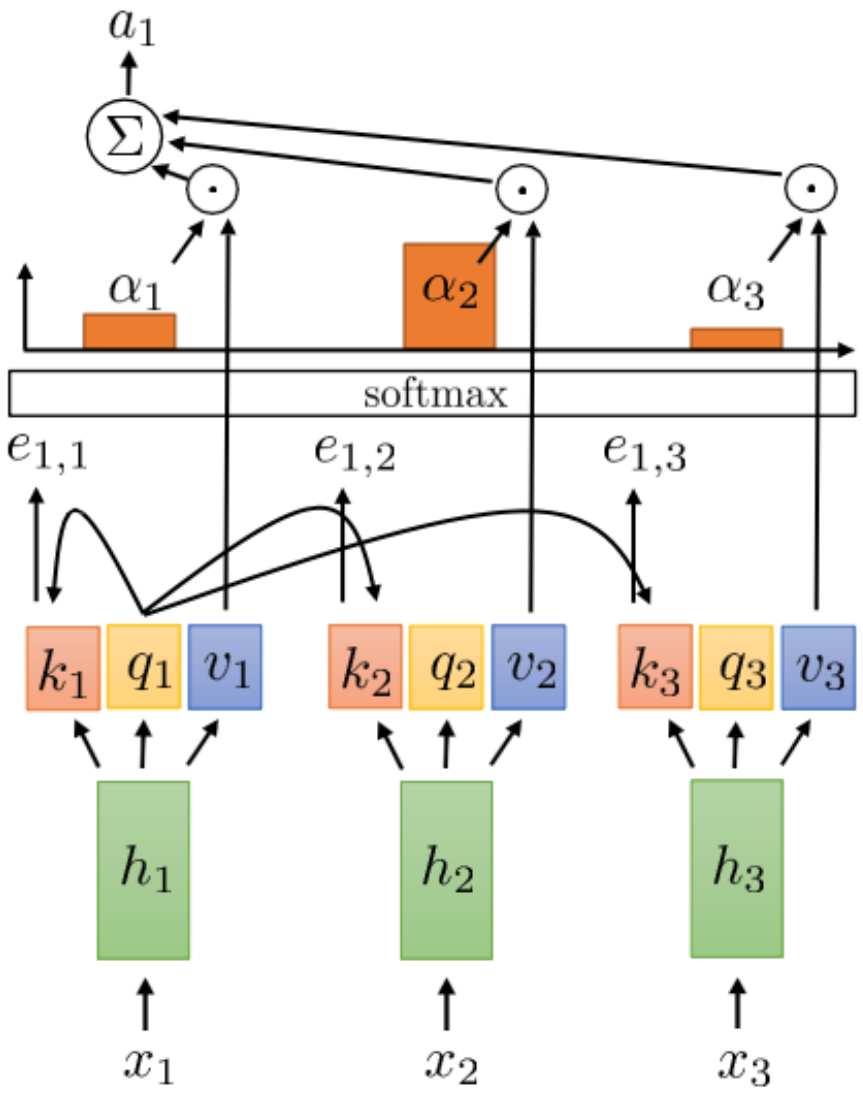


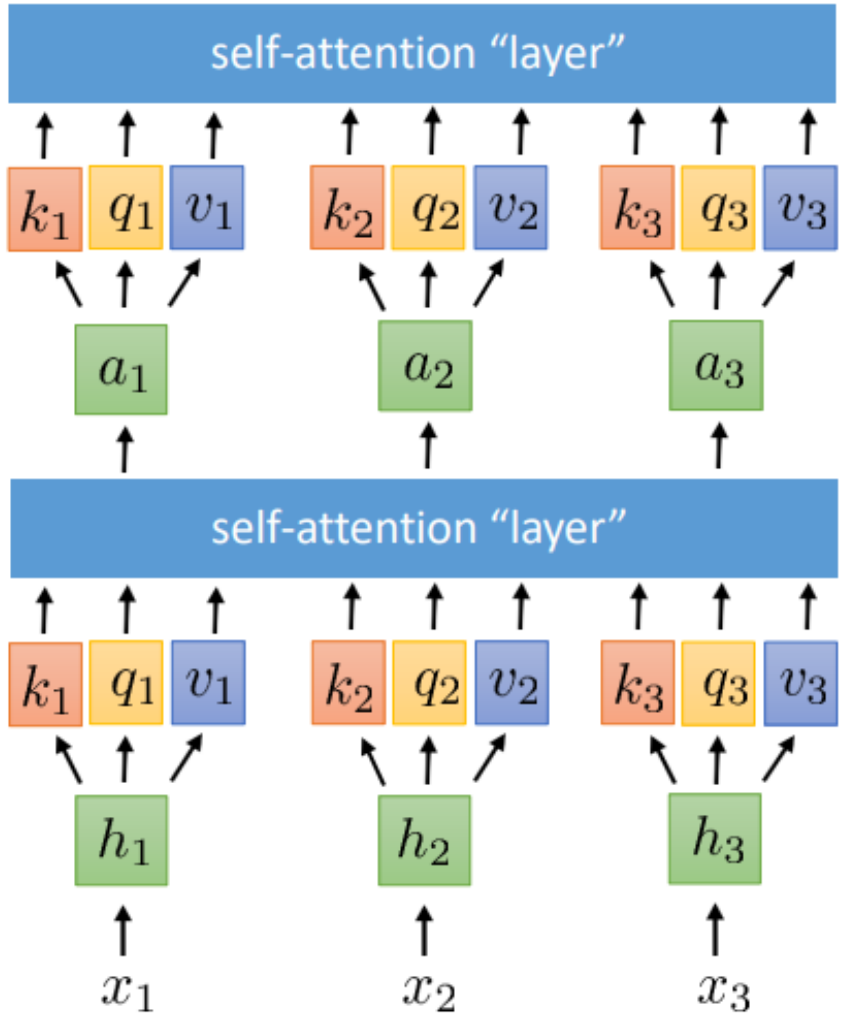


Self-Attention

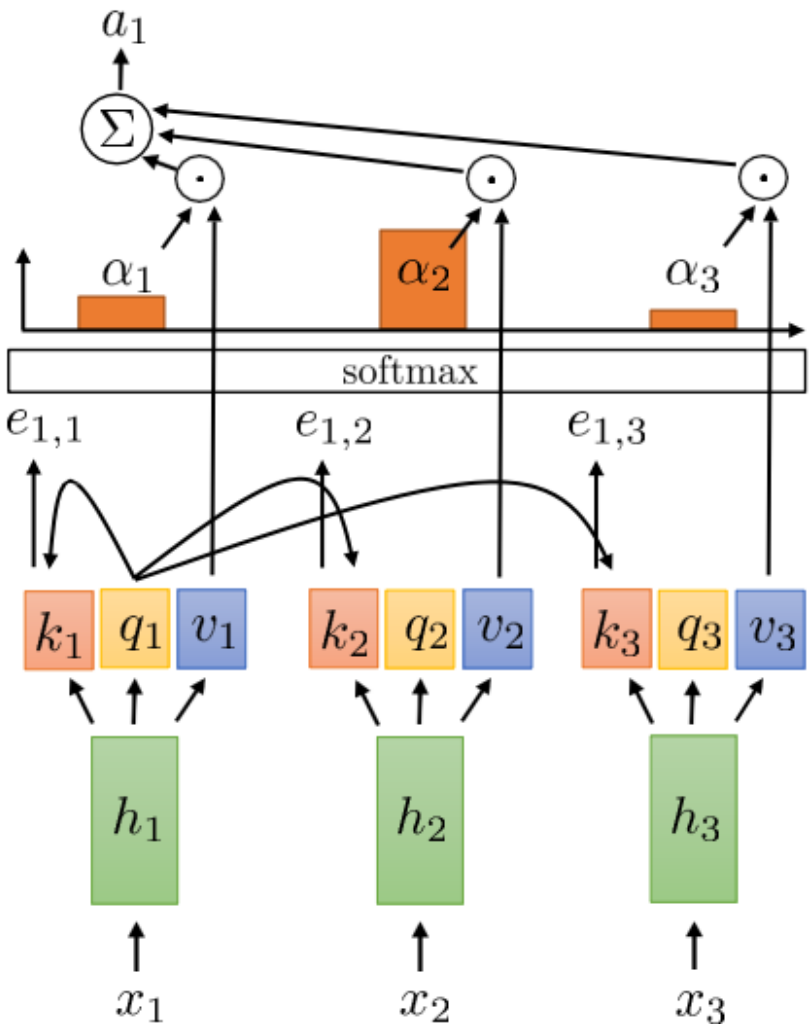


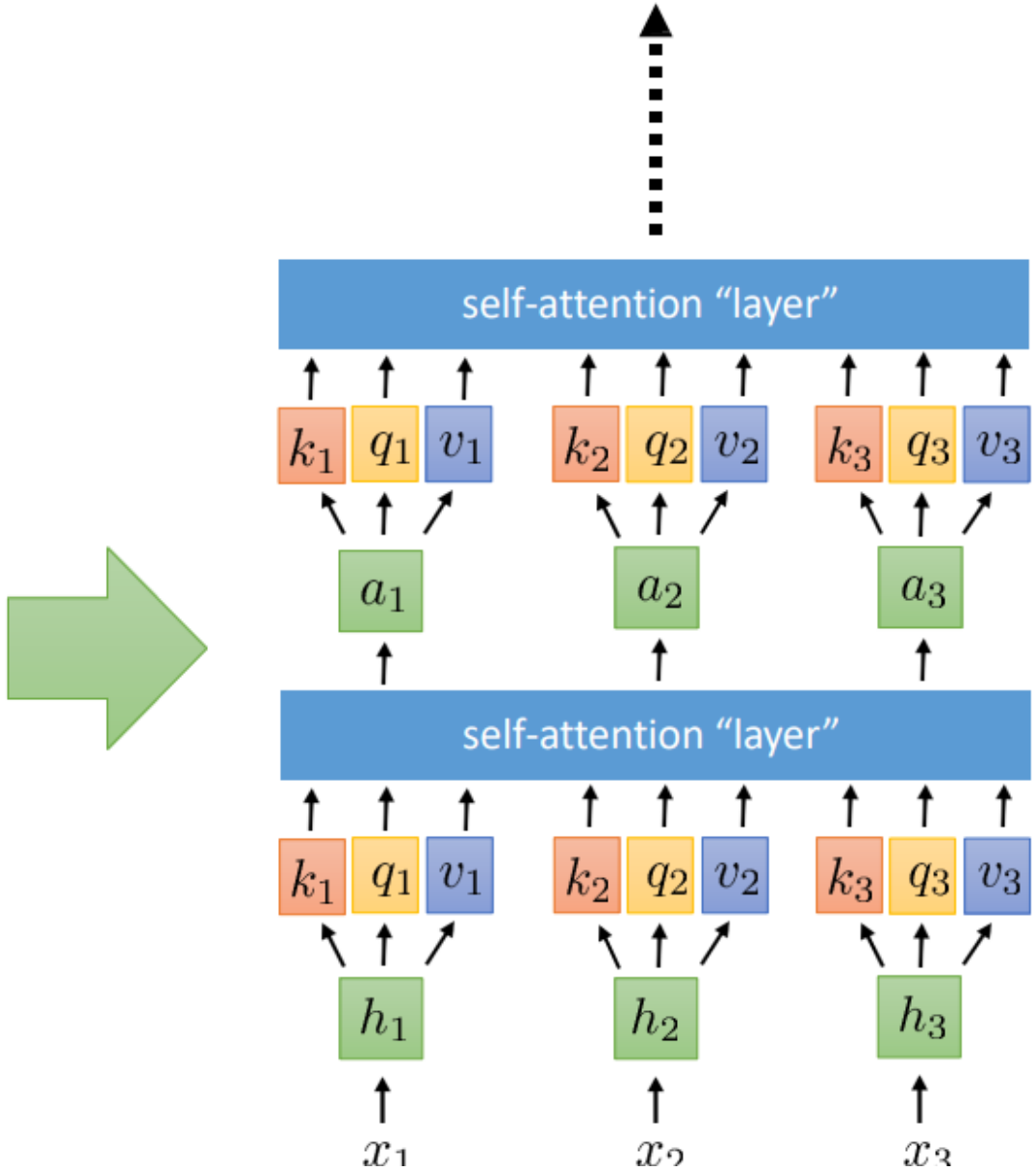
Self-Attention



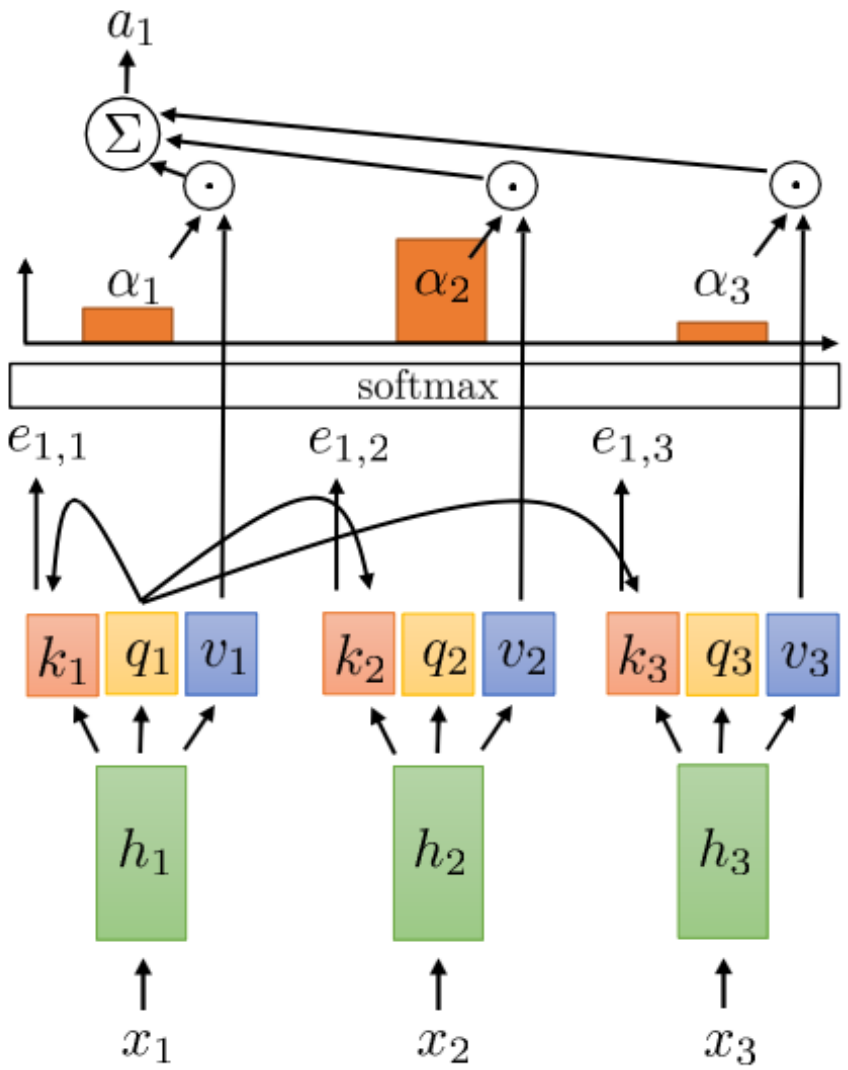


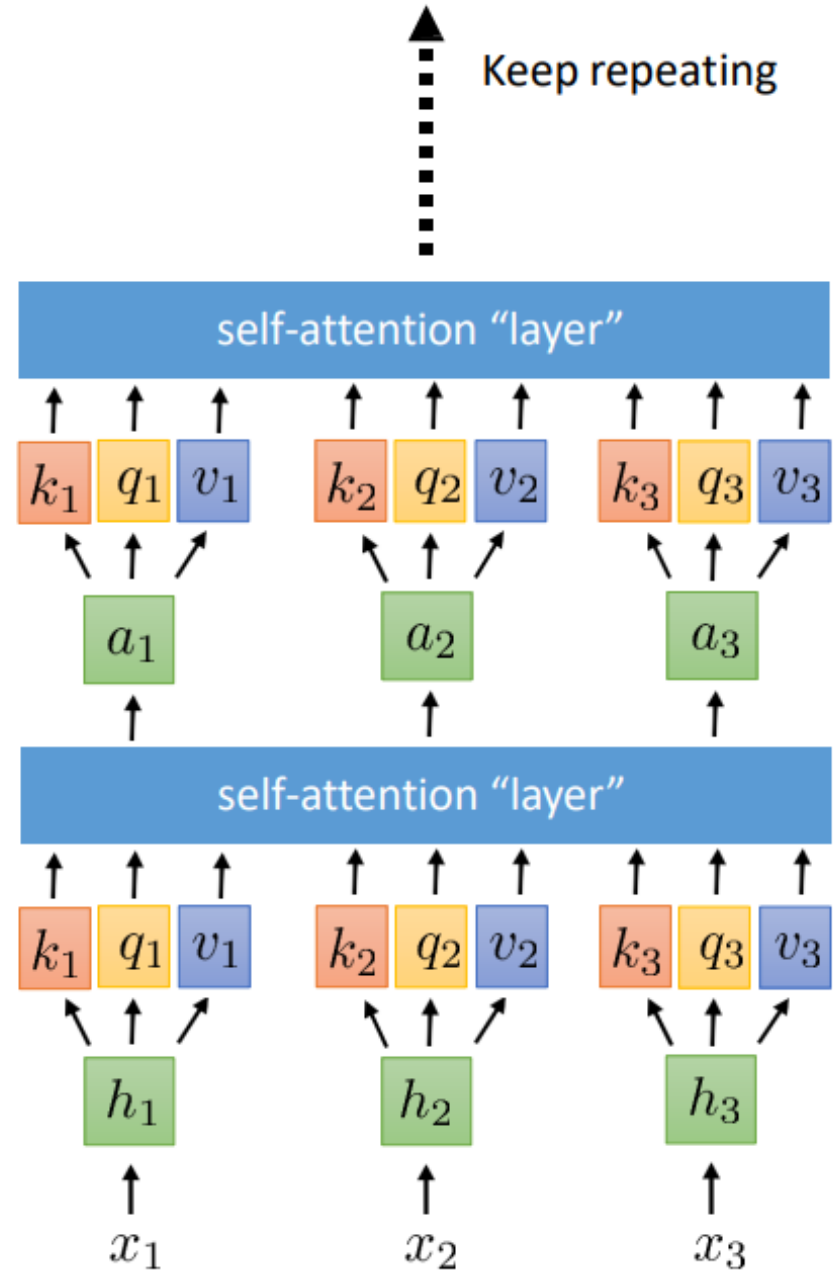
Self-Attention





Self-Attention



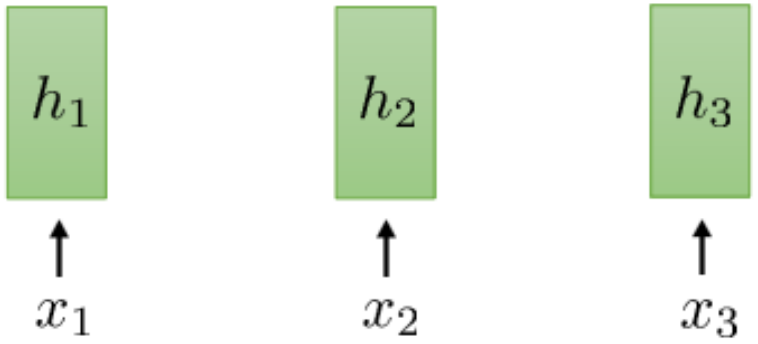


Multi-Head Attention

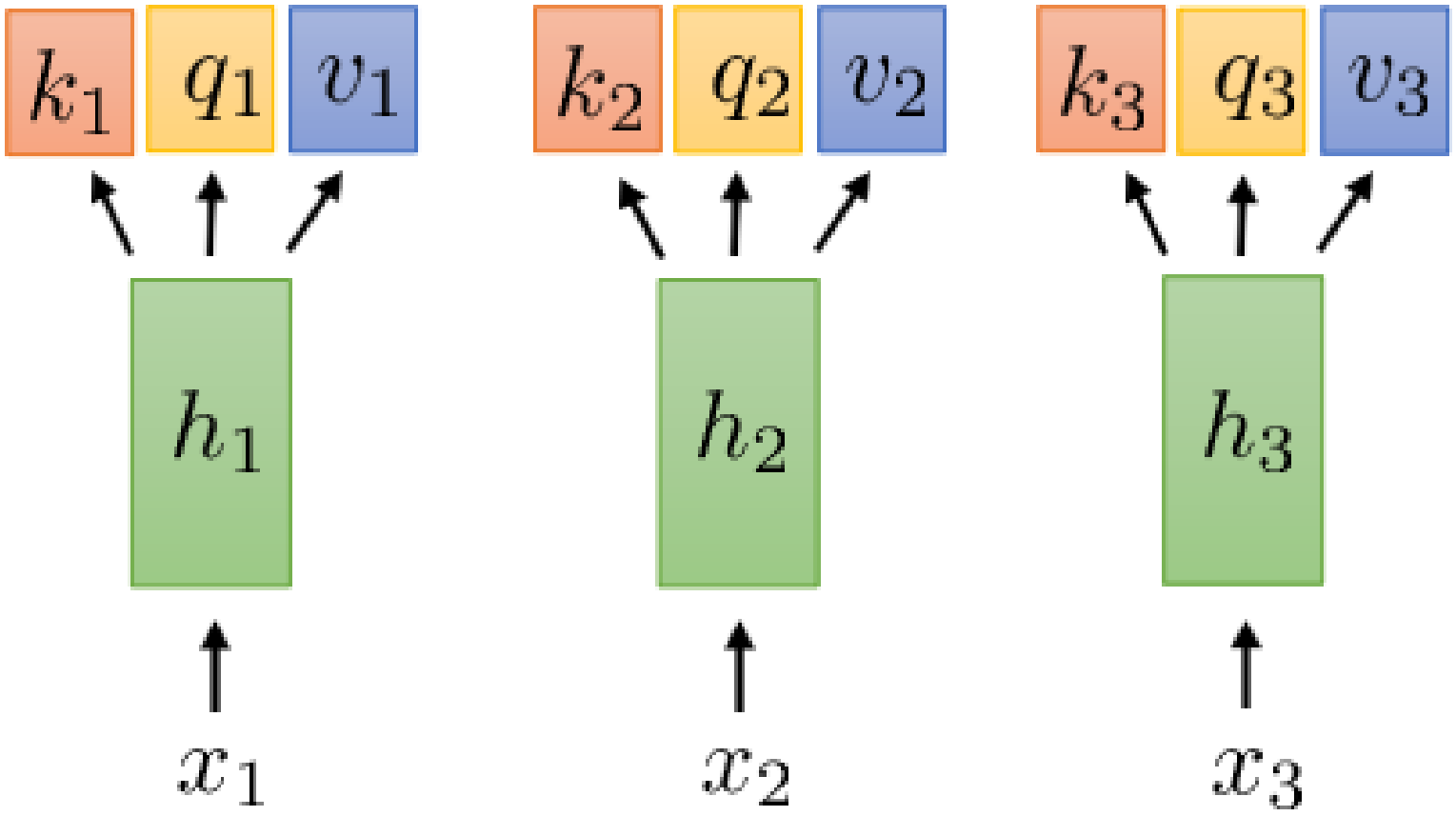
- Multi-Head Attention (MHA) is a mechanism used in Transformer models that allows the model to focus on different parts of an input sequence simultaneously. Each attention head learns different relationships between words or tokens.
- Multi-head attention extends self-attention by splitting the input into multiple heads, enabling the model to capture diverse relationships and patterns. Instead of using a single set of Q, K, V matrices, the input embeddings are projected into multiple sets (heads), each with its own Q, K, V:

Multi-Head Attention

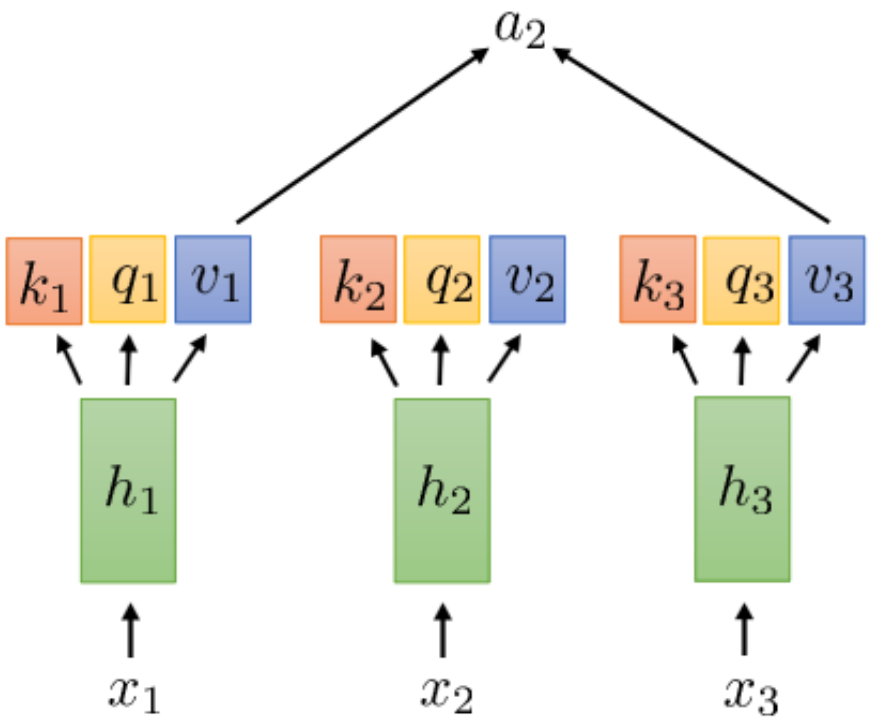
Given that we are fully dependent on attention now it could be beneficial to include more than one time step



Multi-Head Attention



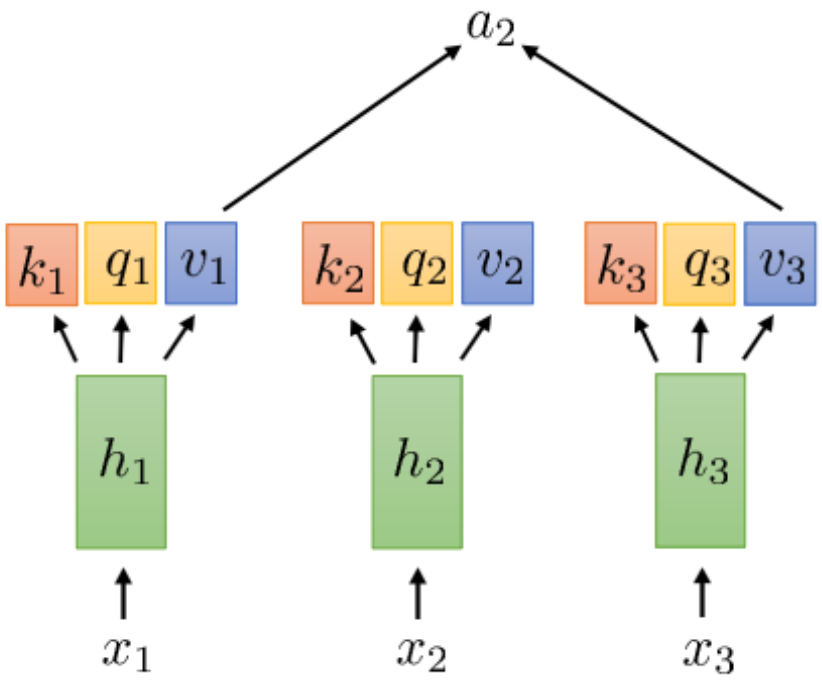
Multi-Head Attention



$$a_l = \sum_t \alpha_{l,t} v_t$$

Due to the softmax function, this will be heavily influenced by a single value.

Multi-Head Attention



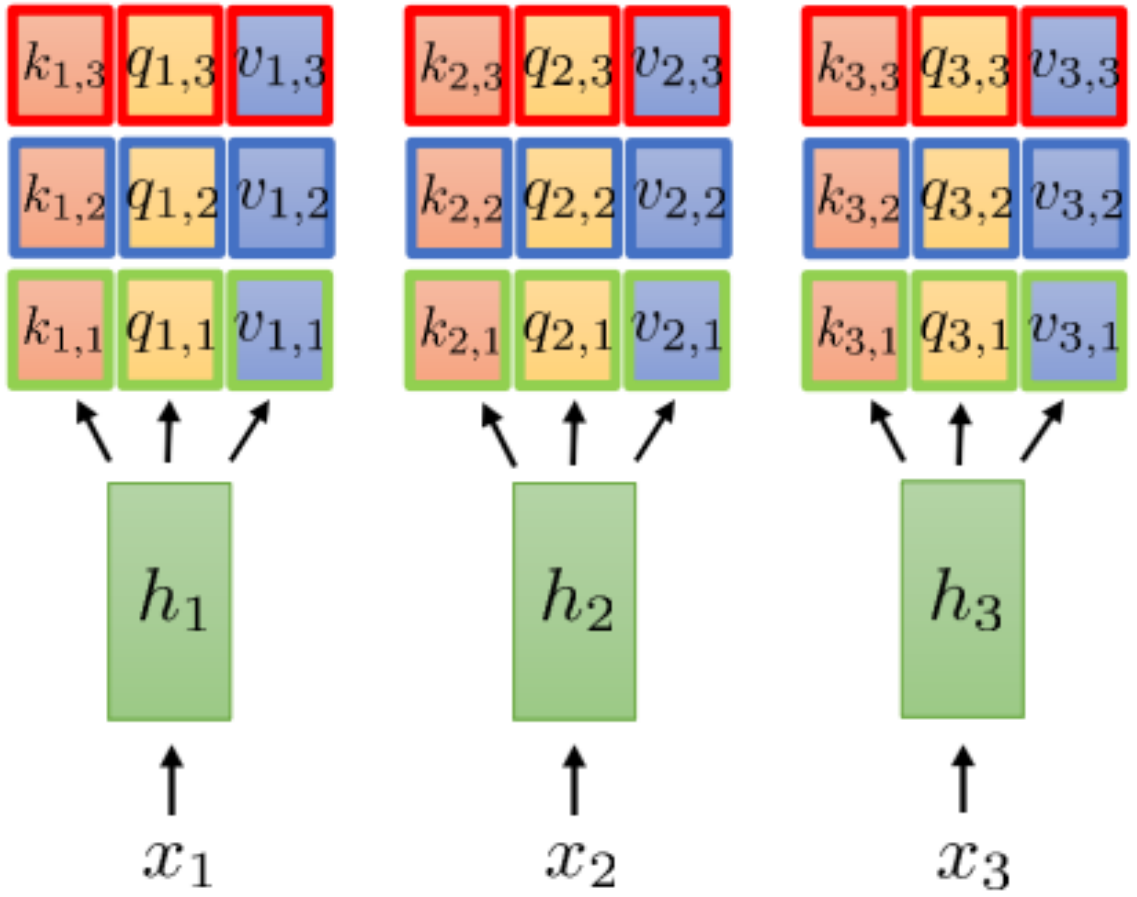
$$a_l = \sum_t \alpha_{l,t} v_t$$

Due to the softmax function, this will be heavily influenced by a single value.

$$e_{l,t} = q_l \cdot k_t$$

It's challenging to clearly specify that you want two distinct elements, like the subject and object in a sentence.

Multi-Head Attention



Multi-Head Attention

Linear Projections: The input X is projected into multiple smaller-dimensional subspaces using different weight matrices. Hidden layer h_1

$$k_{1,3} = W_K^3 h_1; \quad q_{1,3} = W_q^3 h_1; \quad v_{1,3} = W_V^3 h_1$$
$$k_{1,2} = W_K^2 h_1; \quad q_{1,2} = W_q^2 h_1; \quad v_{1,2} = W_V^2 h_1$$
$$k_{1,1} = W_K^1 h_1; \quad q_{1,1} = W_q^1 h_1; \quad v_{1,1} = W_V^1 h_1$$



Multi-Head Attention

Hidden layer h_2

$$k_{2,3} = W_K^3 h_2; \quad q_{2,3} = W_q^3 h_2; \quad v_{2,3} = W_V^3 h_2$$
$$k_{2,2} = W_K^2 h_2; \quad q_{2,2} = W_q^2 h_2; \quad v_{2,2} = W_V^2 h_2$$
$$k_{2,1} = W_K^1 h_2; \quad q_{2,1} = W_q^1 h_2; \quad v_{2,1} = W_V^1 h_2$$



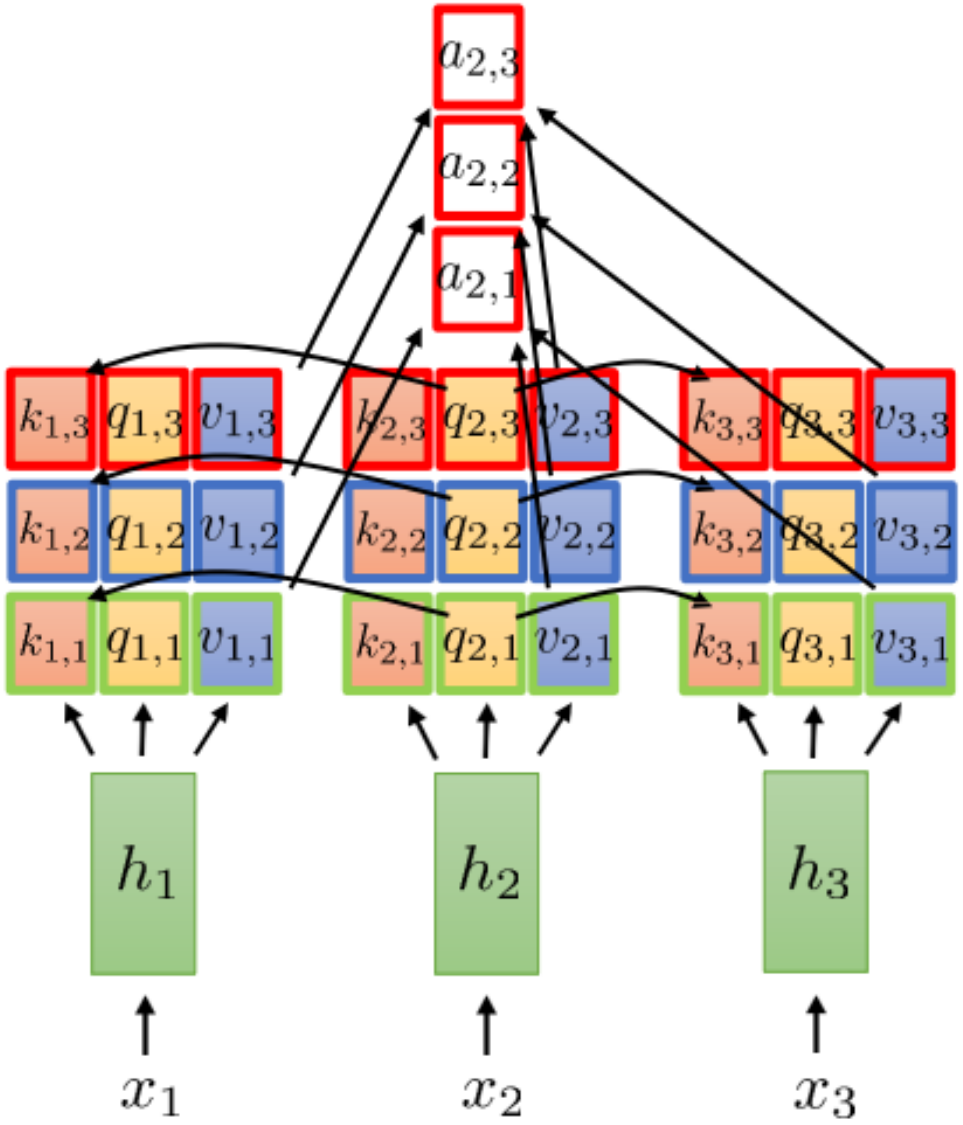
Multi-Head Attention

Hidden layer h_3

$$k_{3,3} = W_K^3 h_3; \quad q_{3,3} = W_q^3 h_3; \quad v_{3,3} = W_V^3 h_3$$
$$k_{3,2} = W_K^2 h_3; \quad q_{3,2} = W_q^2 h_3; \quad v_{3,2} = W_V^2 h_3$$
$$k_{3,1} = W_K^1 h_3; \quad q_{3,1} = W_q^1 h_3; \quad v_{3,1} = W_V^1 h_3$$



Multi-Head Attention



after concatenation

$$a_1 = \begin{bmatrix} a_{1,1} \\ a_{1,2} \\ a_{1,3} \end{bmatrix} \quad a_2 = \begin{bmatrix} a_{2,1} \\ a_{2,2} \\ a_{2,3} \end{bmatrix} \quad a_3 = \begin{bmatrix} a_{3,1} \\ a_{3,2} \\ a_{3,3} \end{bmatrix}$$

attention output

$$a_{l,i} = \sum_{t'} \alpha_{l.t,i} \, v_{t,i}$$

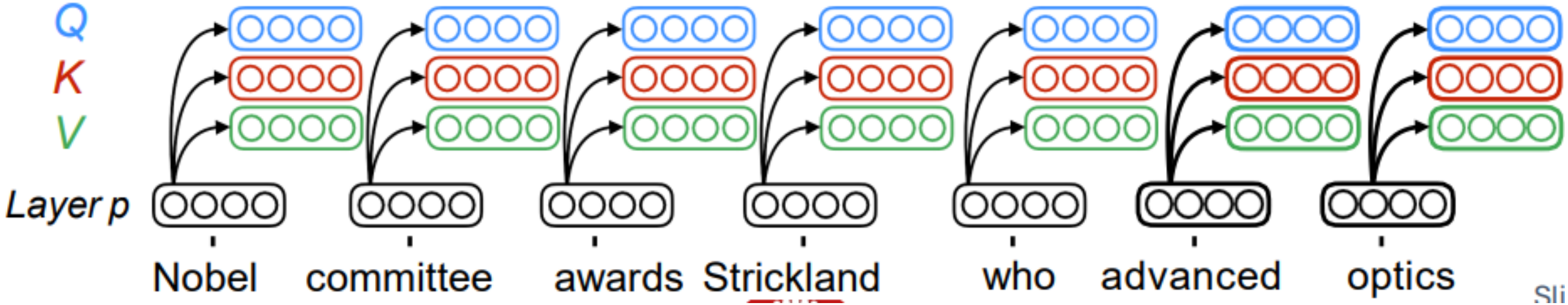
attention distribution

$$\alpha_{l,t,i} = softmax(e_{l,t,i}) = \frac{\exp(e_{l,t,i})}{\sum_{t'} \exp(e_{l,t,i})}$$

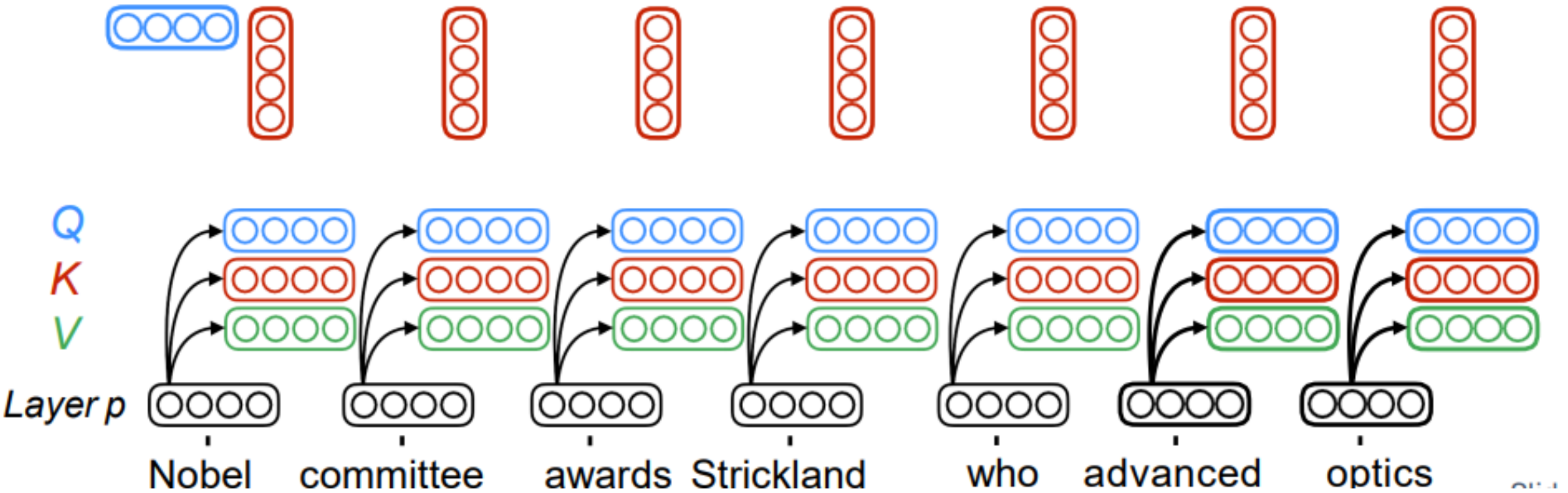
attention Score

$$e_{l,t,i} = q_{l,i} k_{l,i}$$

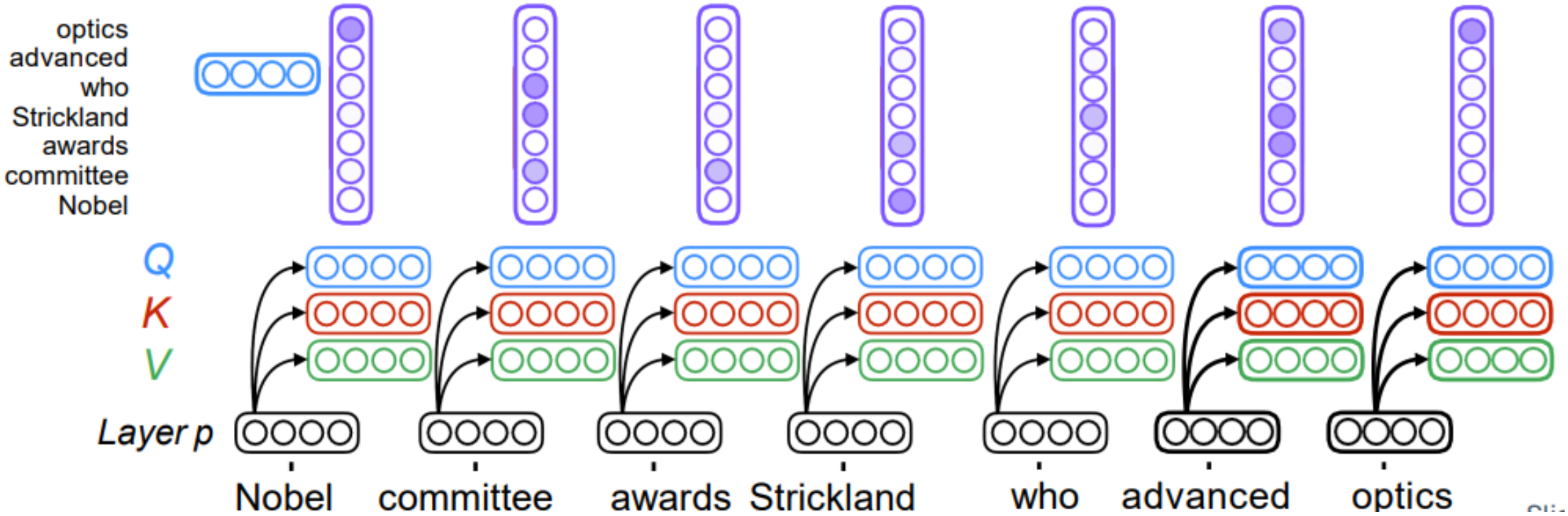
Self Attention (In Encoder)



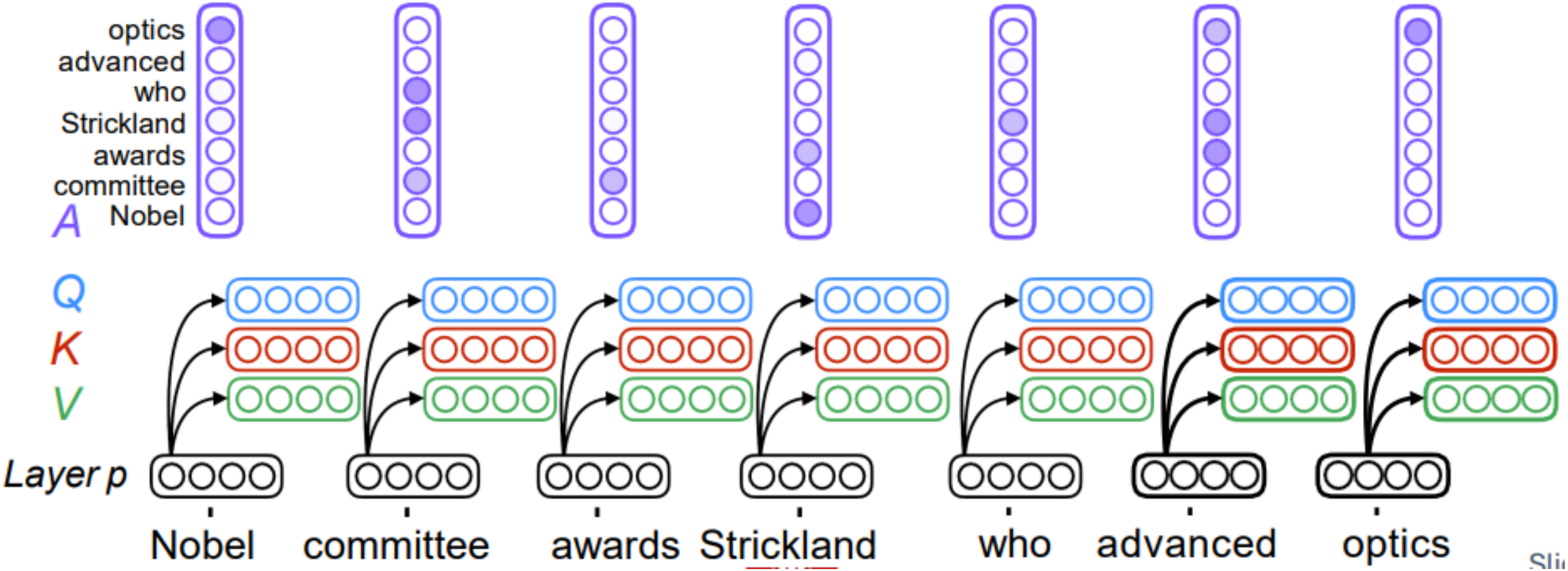
Self Attention (In Encoder)



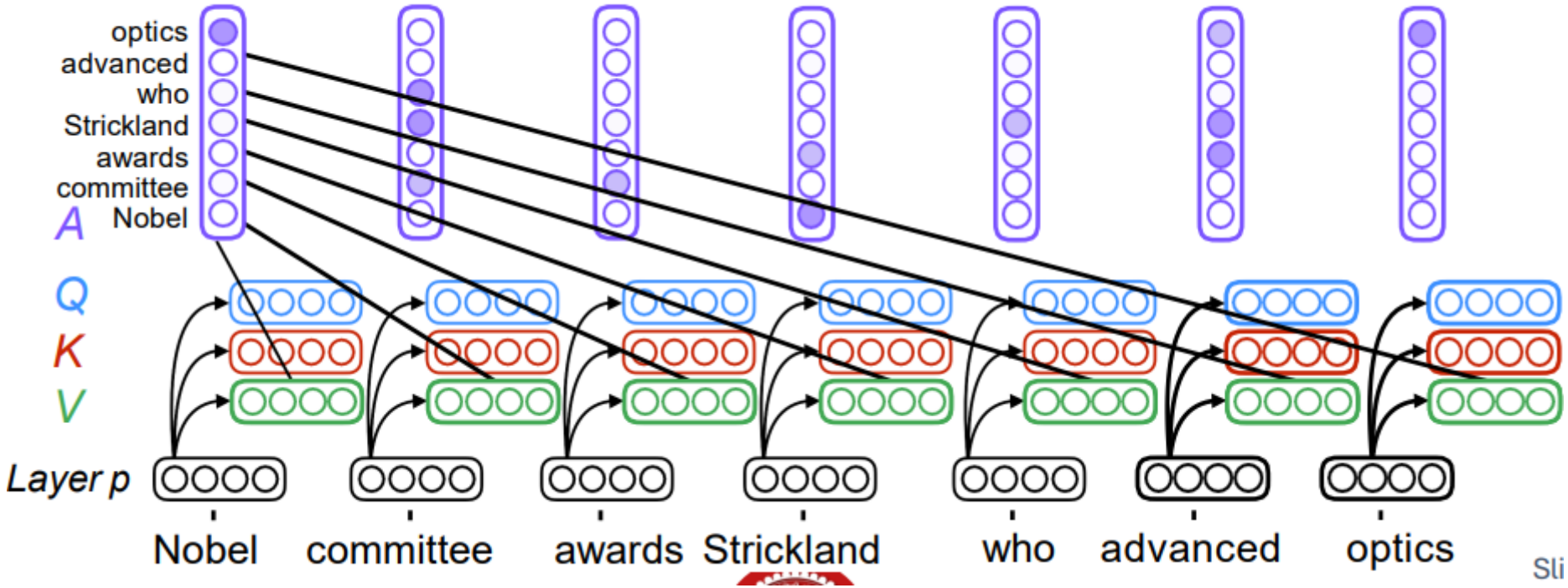
Self Attention (In Encoder)



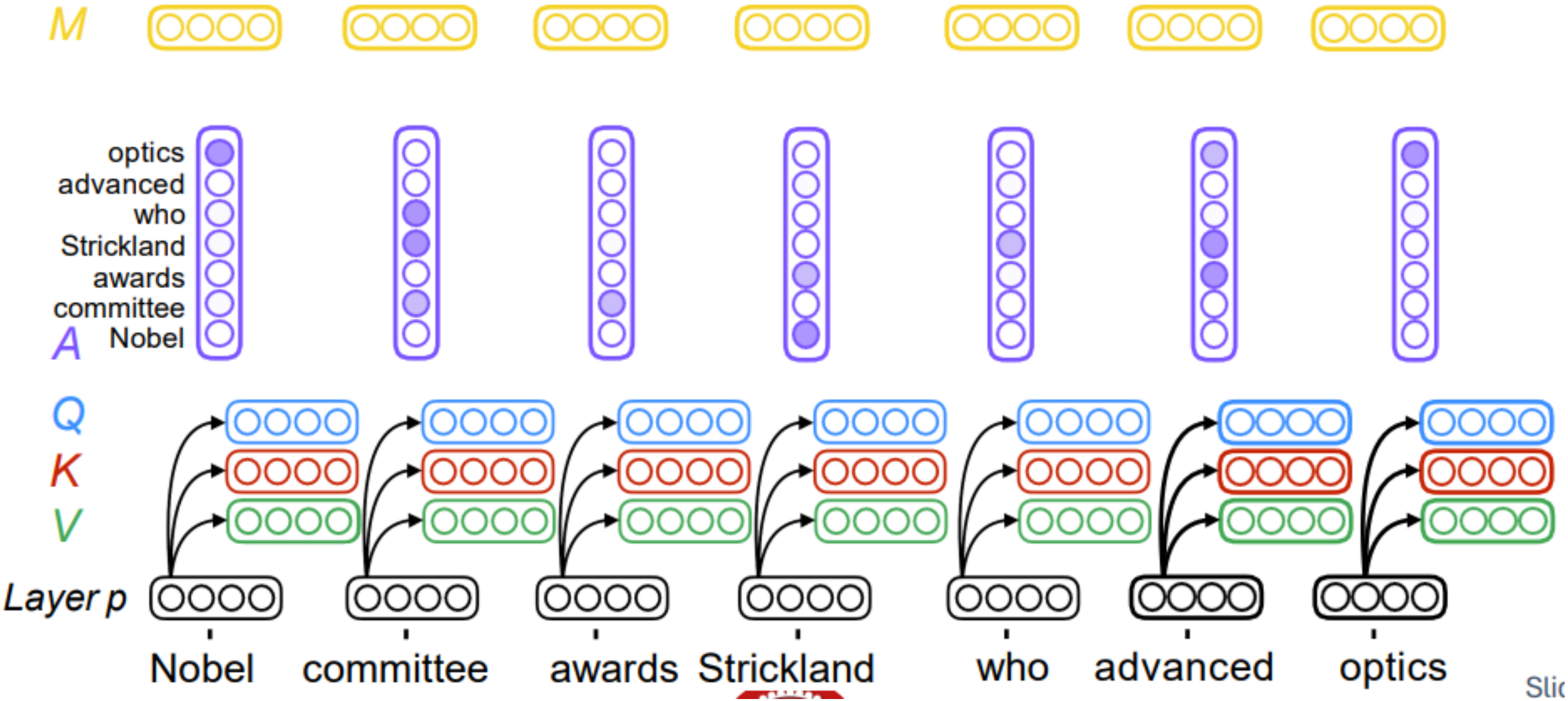
Self Attention (In Encoder)



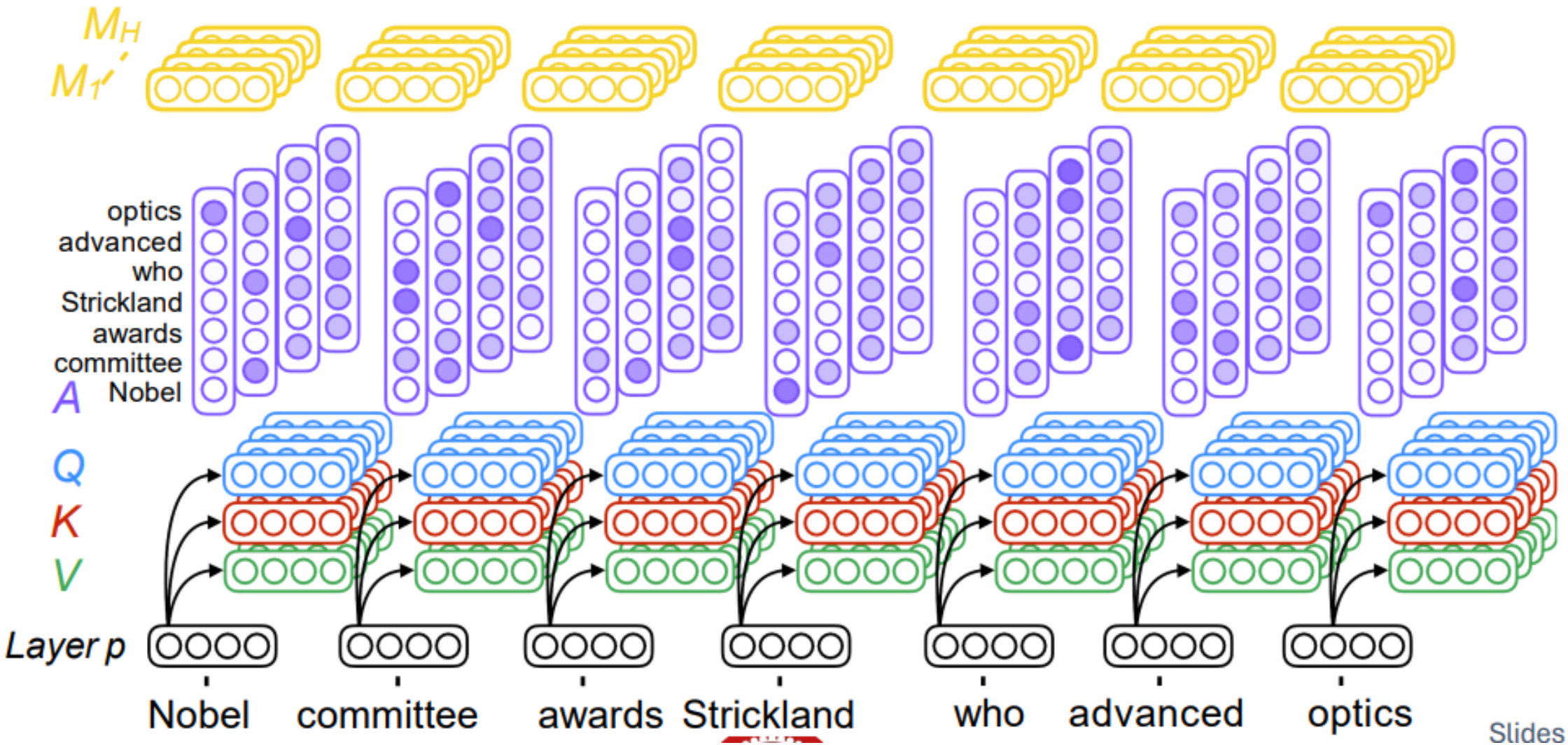
Self Attention (In Encoder)



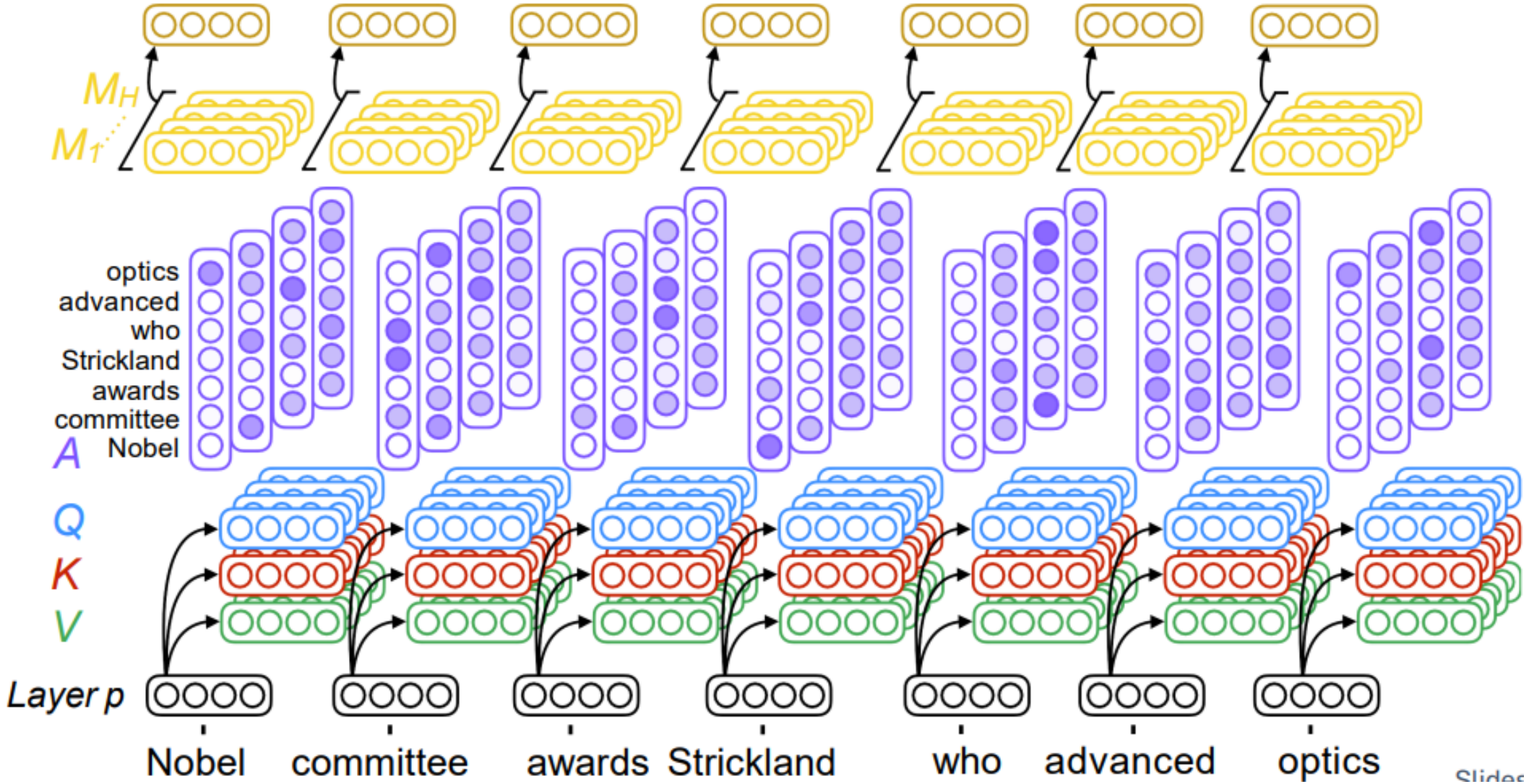
Self Attention (In Encoder)



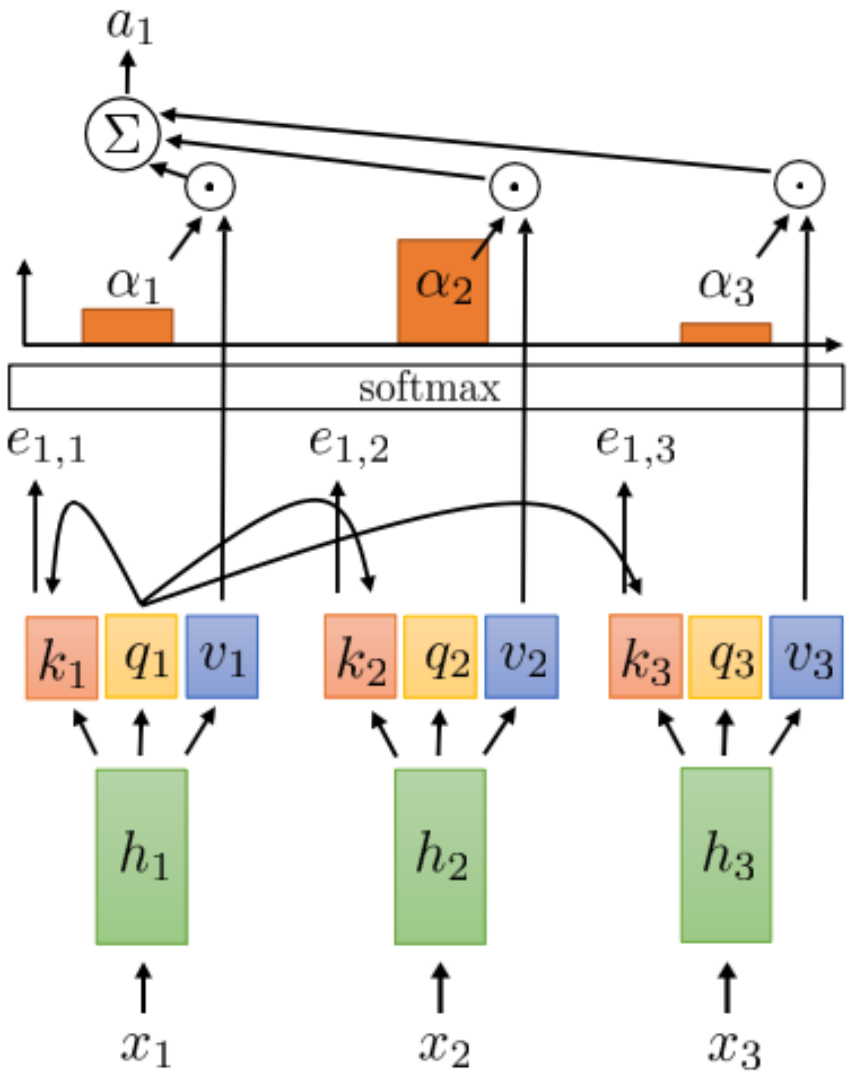
Multi Head Self Attention



Multi Head Self Attention



Self Attention is linear



$$k_t = W_k h_t \qquad q_t = W_q h_t \qquad v_t = W_v h_t$$

$$\alpha_{l,t} = \exp(e_{l,t}) / \sum_{t'} \exp(e_{l,t'})$$

$$e_{l,t} = q_l \cdot k_t$$

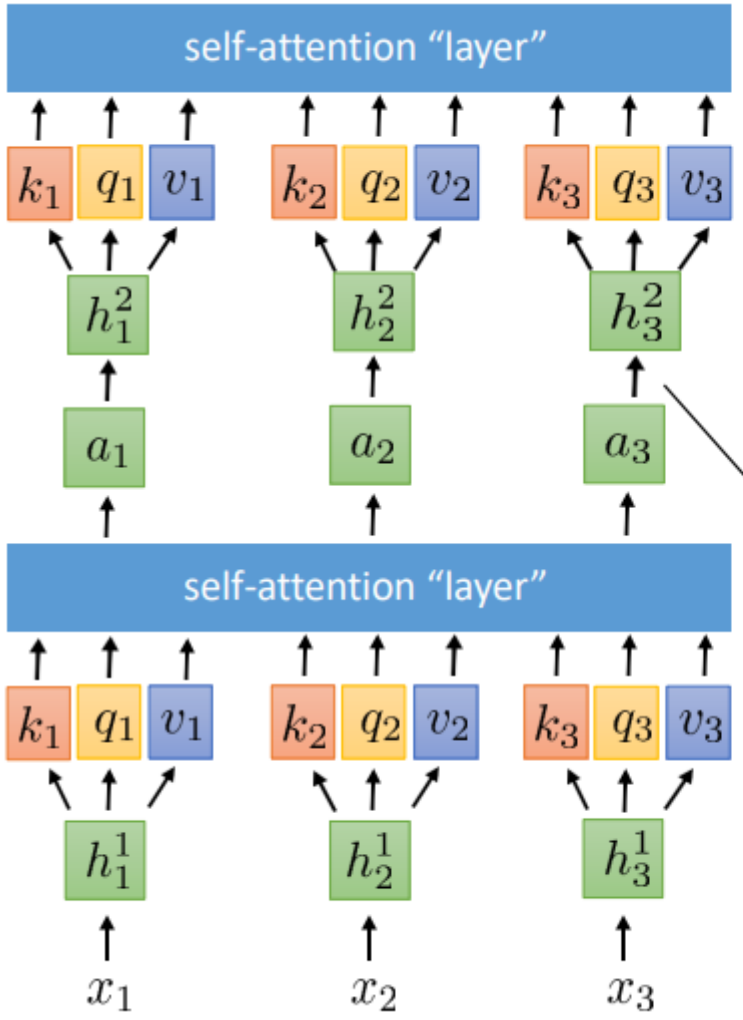
$$a_l = \sum_t \alpha_{l,t} v_t = \sum_t \alpha_{l,t} W_v h_t = W_v \sum_t \alpha_{l,t} h_t$$

linear transformation

non-linear weights

Problem: Every self-attention layer is a linear transformation of the previous layer with non-linear weights.

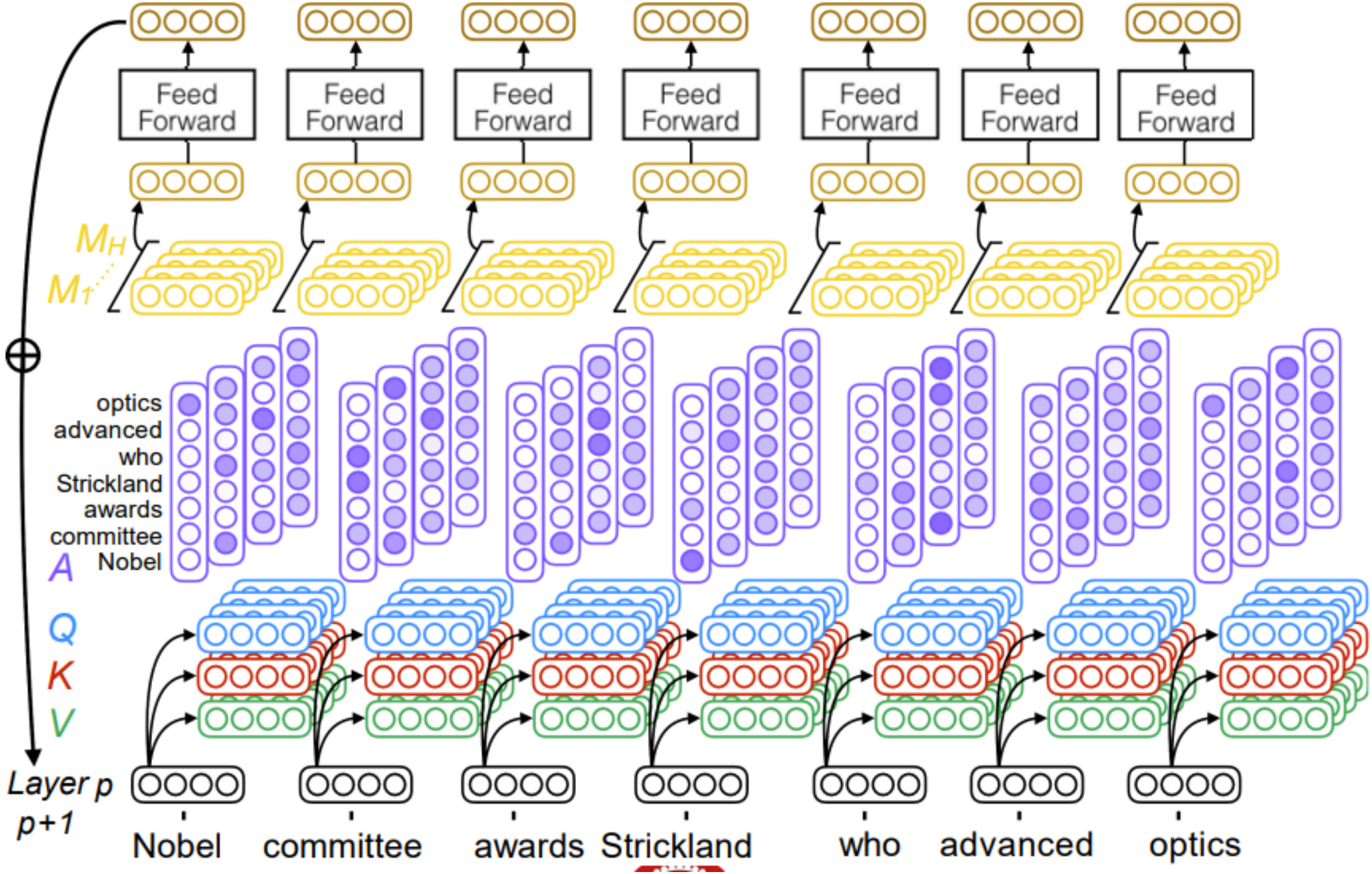
Position-wise Feed-Forward Networks



- **Solution :** Make the model more expressive is by alternating use of self-attention and non-linearity.
- Non-linearity is incorporated by means of a feed-forward network which consists of two linear transformations with a ReLU activation in between.

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

- The same non-linearity is utilized across various positions but they differ from layer to layer.


Feed Forward
Feed Forward
Feed Forward
Feed Forward
Feed Forward
Feed Forward
Feed Forward
M_H
M_1
optics
advanced
who
Strickland
awards
committee
Nobel
A
Q
K
V
Layer p
p+1
Nobel
committee
awards
Strickland
who
advanced
optics